Universidad Nacional de Córdoba
Facultad de Ciencias Exactas, Físicas y Naturales

FCEFyN

Procesamiento Digital de Señales

Ingeniería en Computación

Laboratorio 2

Filtros FIR con CMSIS-DSP en MCXN947

FRDM-MCXN947 · MCUXpresso IDE · CMSIS-DSP · MATLAB

Integrante	**Matrícula**	**Correo**
Costamagna, Matias Javier	44.512.437	matias.costamagna@mi.unc.edu.ar
Moreyra, Julián	45.214.537	julian.moreyra@mi.unc.edu.ar
Rivera, Mariano	45.432.921	luismarianorivera.25@mi.unc.edu.ar
Rojas, Facundo Nicolás	44.629.222	facundo.rojas.222@mi.unc.edu.ar

Docentes: Rossi, Roberto Raúl · Parlanti, Gustavo Fernando · Molina, Germán Alcides · Ferreyra, Pablo Alejandro

Fecha de entrega: 29 de octubre de 2026

Índice

1. Introducción **3**
1.1. Requerimientos . . . 3

2. Configuración del ambiente **4**
2.1. Herramientas utilizadas . . . 4
2.2. Creación del proyecto . . . 4
2.3. Relojes y pines . . . 4
2.4. Cambios en los periféricos . . . 5

3. Diseño de los filtros **7**
3.1. Especificaciones . . . 7
3.2. Métodos de diseño . . . 7
3.3. Implementación previa: un FIR directo por velocidad . . . 9
3.4. Implementación multitasa . . . 14
3.5. De MATLAB al firmware . . . 20
3.6. Comparación con la implementación previa . . . 21

4. Diseño de la aplicación **23**
4.1. Arquitectura general . . . 23
4.2. Velocidades de muestreo . . . 23
4.3. Máquina de estados . . . 24
4.4. Habilitación del filtro e indicación en el LED . . . 24
4.5. Formato q15 y conversiones . . . 26
4.6. Filtrado . . . 27
4.7. Rutinas de interrupción . . . 28
4.8. Restricción de tiempo real . . . 30

5. Implementación **33**
5.1. Constantes y variables globales . . . 33
5.2. Programa principal . . . 33
5.3. Inicialización de los filtros . . . 33
5.4. Rutina del ADC . . . 33
5.5. Rutinas de las teclas y cambio de velocidad . . . 35

6. Verificación **36**
6.1. Procedimiento . . . 36
6.2. Verificación por simulación . . . 36
6.3. Respuesta en frecuencia medida . . . 36

6.4. Cumplimiento de los requerimientos 40

7. Conclusiones **41**

A. Código fuente **42**

1 Introducción

Laboratorio 2 — Filtros FIR por muestras

Objetivo. Realizar el diseño e implementación de filtros FIR usando procesamiento por muestras.

Descripción. Realizar un programa aplicativo que sea capaz de aplicar un filtro FIR, por muestras, a un buffer de memoria de 512 muestras, que es adquirido con el Laboratorio 1 teniendo en cuenta las frecuencias de muestreo de 8 kHz, 16 kHz, 22 kHz, 44 kHz y 48 kHz. Con una de las teclas de la placa de evaluación FRDM-MCXN947 se habilitará la aplicación del filtro o se hará *bypass* del filtro a un buffer de salida distinto del buffer de entrada. De este buffer de salida se enviarán las muestras al DAC de la placa. Visualizar el resultado utilizando un osciloscopio y comparar los resultados.

Requerimientos de los filtros.

F1 Pasa bajos: $f_c = 3\,600$ Hz, $A_{\text{stop}} = 30$ dB.
F2 Pasa altos: $f_c = 35$ Hz, $A_{\text{stop}} = 30$ dB.
F3 Pasa banda: $f_{c1} = 35$ Hz, $f_{c2} = 3\,500$ Hz, $A_{\text{stop}} = 30$ dB.
F4 Elimina banda: $f_r = 50$ Hz, $BW = 15$ Hz, $A_{\text{stop}} = 25$ dB.

1.1 Requerimientos

De la consigna se desprenden los siguientes requerimientos, que se usan como guía para el diseño y como lista de verificación en la sección 6:

R1 Adquirir la señal con la aplicación del Laboratorio 1, a 8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s, en un buffer de 512 muestras.

R2 Diseñar, para cada frecuencia de muestreo, filtros FIR que cumplan las especificaciones **F1** a **F4**.

R3 Aplicar el filtro por muestras al buffer de entrada, escribiendo el resultado en un buffer de salida distinto.

R4 Con una tecla, habilitar el filtro o hacer *bypass* del buffer de entrada al de salida.

R5 Enviar las muestras del buffer de salida al DAC.

R6 Visualizar la entrada y la salida con un osciloscopio y comparar los resultados.

Diferencias respecto de la consigna

La consigna pide aplicar un filtro y, con una tecla, habilitarlo o hacer *bypass* a un buffer de salida. La implementación:

- recorre con SW2 los cuatro filtros **F1**–**F4**, para compararlos sobre la misma señal sin reprogramar la placa;
- habilita o deshabilita el filtro con el touchpad capacitivo E1 de la placa en lugar de un pulsador; en *bypass* copia la muestra de entrada al buffer de salida, del que el DAC lee siempre;
- indica el filtro activo con el parpadeo del LED (sección 4.4);
- implementa el PA, el PBanda y el EB con una estructura multitasa (sección 3.4): siguen siendo FIR y se procesan muestra a muestra, pero no como un único FIR directo.

El filtrado trabaja en q15, con el mismo formato del buffer del Laboratorio 1: `arm_fir_q15()` de CMSIS-DSP para el PB, el LPB y los filtros G, y código propio para el diezmador y el interpolador.

2 Configuración del ambiente

2.1 Herramientas utilizadas

Tabla 1: Hardware y software del entorno de trabajo.

Elemento	Versión / modelo
Placa de evaluación	FRDM-MCXN947 (Cortex-M33, núcleo `cm33_core0`)
IDE	MCUXpresso IDE v25.6.136
SDK	`SDK_26.06.00_FRDM-MCXN947`
Herramientas de config.	MCUXpresso Config Tools (Clocks, Pins, Peripherals)
Proyecto base	Proyecto del Laboratorio 1 (`TP1`)
Bibliotecas	Drivers del SDK (incluido `fsl_tsi_v6`) y CMSIS-DSP (`arm_fir_q15`)
Diseño de filtros	MATLAB R2023a (9.14.0.2206163), `firpm`, `fdatool`
Generador de señales	
Osciloscopio	

2.2 Creación del proyecto

El proyecto `TP2` parte del proyecto del Laboratorio 1: conserva la cadena CTIMER0 → INPUTMUX → LPADC0 → DAC0, la selección circular de la frecuencia de muestreo y la indicación con el LED. Sobre esa base:

1. Se agregó al proyecto el código fuente de CMSIS-DSP (carpeta `CMSIS/DSP`) para disponer de `arm_fir_init_q15()` y `arm_fir_q15()`.
2. Los filtros se diseñaron en MATLAB con el script `disenar_multitasa.m`, que escribe los coeficientes y los parámetros de la estructura multitasa en el archivo de cabecera `source/filtros_multitasa.h` (sección 3). La implementación previa usaba `fdatool` y el script `generar_filtros.m`, que generaba `source/filtros.h` (sección 3.3).
3. Se agregó el componente `tsi_v6` del SDK (*Manage SDK Components*), con el driver `fsl_tsi_v6`, para leer el touchpad de la placa.
4. La lógica de la aplicación se escribió en `source/TP2.c`.
5. La depuración se hace con el MCU-Link integrado de la placa, con la configuración guardada en `TP2 LinkServer Debug.launch`.

Agregar componentes del SDK reemplaza los archivos de la placa

Al agregar el componente `tsi_v6`, MCUXpresso reemplazó los archivos generados de `board/` (`pin_mux`, `clock_config` y `peripherals`) por los de la plantilla de la placa: desaparecieron las inicializaciones del CTIMER0, el LPADC0, el DAC0 y los GPIO. La configuración no se pierde, porque está en `TP2.mex`; basta volver a generar el código desde Config Tools (*Update Code*). Conviene revisar los cambios de `board/` antes de compilar después de agregar un componente.

2.3 Relojes y pines

Los relojes y la asignación de pines son los mismos del Laboratorio 1 (PLL0 a 150 MHz para el núcleo y el CTIMER0, FRO_HF de 48 MHz para el ADC0 y el DAC0). La tabla 2 los resume; las capturas de Config Tools están en el informe del Laboratorio 1.

Tabla 2: Asignación de pines (sin cambios respecto del Laboratorio 1, salvo la función de SW2).

Función	Pin	Periférico	Conector / observación
Entrada analógica	`ADC0_A0`	LPADC0 canal A0	`J4[2]` (Arduino A0), 0 V a 3,3 V
Salida analógica	`P4_2`	DAC0_OUT	`J1[4]`
Disparo del ADC	`P0_19`	CT0_MAT3	`J2[13]`, punto de prueba
LED rojo / verde / azul	`P0_10`, `P0_27`, `P1_2`	GPIO0, GPIO1	Activos en bajo
Tecla SW3 (velocidad)	`P0_6`	GPIO0, canal IRQ 0	Flanco descendente, pull-up interno
Tecla SW2 (filtro)	`P0_23`	GPIO0, canal IRQ 1	Flanco descendente
Touchpad E1 (filtrado)	`P1_3`	TSI0, canal 3	Self-cap, disparo por software
Medición de la ISR	`P0_26`	GPIO0, salida	`J2[10]`, en 1 durante la ISR del ADC

2.4 Cambios en los periféricos

La tabla 3 resume lo que cambió respecto del Laboratorio 1. El resto de la configuración (CTIMER0 en modo toggle sobre el match 3, disparo del ADC por INPUTMUX, interrupciones de las teclas en las líneas 0 y 1 del GPIO0) se mantiene.

Tabla 3: Diferencias de configuración respecto del Laboratorio 1.

Elemento	Laboratorio 1	Laboratorio 2
Referencia del LPADC0	`kLPADC_ReferenceVoltageAlt1`	`kLPADC_ReferenceVoltageAlt3`
Calibración del ADC	En `main()`	En `ADC0_init()` (generado por Config Tools)
Referencia del DAC0	`kDAC_ReferenceVoltageSourceAlt3`	`kDAC_ReferenceVoltageSourceAlt1`, como el ejemplo `dac_basic` del SDK
Inicialización del DAC0	Config Tools	`init_dac()` antes de `BOARD_InitBootPeripherals()`
Tecla SW2	Run/Stop	Selección de filtro (PB, PA, PBanda, EB)
Habilitación del filtro	—	Touchpad E1 (TSI0, canal 3, `P1_3`), leído desde el lazo principal. Pin y reloj del TSI (FRO de 12 MHz) en Config Tools
LED RGB	Color según la f_s	Color según la f_s; parpadeo según el filtro, fijo con la salida sin filtrar
Lazo principal	Vacío	Lee el touchpad y controla el LED, con una base de tiempo de 1 ms (SysTick)
Pin de medición	—	`P0_26` en 1 mientras se ejecuta la rutina del ADC (`MEDIR_TIEMPO_ISR`)
Buffers	`samples[512]`	`buffer_in[512]` y `buffer_out[512]`

El LPADC0 trabaja en resolución alta (16 bit), así que su resultado, centrado en cero, ocupa directamente todo el rango de un `q15_t` (sección 4.5).



Figura 1: Config Tools — ADC0. Comando de conversión del LPADC0 en resolución alta (16 bit).

2.4.1 Orden de inicialización

`BOARD_InitBootPeripherals()` arranca el CTIMER0 y habilita la interrupción del ADC0, así que a partir de ese momento la rutina del ADC empieza a ejecutarse y a escribir en el DAC. Por eso `main()` llama a `init_dac()` (reloj, alimentación desde el SPC e inicialización del DAC0) *antes* de inicializar los periféricos. Los filtros, en cambio, se inicializan después: mientras `filtros_listos` sea falso, la rutina del ADC hace *bypass*, es decir, copia la muestra al buffer de salida sin filtrarla.

2.4.2 Prioridades de interrupción

Igual que en el Laboratorio 1, las tres interrupciones (`ADC0_IRQn`, `GPIO00_IRQn` y `GPIO01_IRQn`) quedan con la prioridad por defecto (0), así que ninguna desaloja a otra. Esto es lo que permite que el cambio de filtro se aplique de forma segura (sección 4.6.5). A diferencia del Laboratorio 1, la rutina del ADC ahora puede ser larga (ejecuta el filtro completo), y su duración frente al período de muestreo se analiza en la sección 4.8. El SysTick, que sólo cuenta milisegundos para el touchpad y el LED (sección 4.4), queda con la prioridad mínima, así que nunca demora a la rutina del ADC.

3 Diseño de los filtros

3.1 Especificaciones

Un filtro FIR de N coeficientes calcula cada muestra de salida como

$$y[n] = \sum_{k=0}^{N-1} b_k \, x[n-k]. \tag{1}$$

Con coeficientes simétricos ($b_k = b_{N-1-k}$) la fase es lineal y todas las frecuencias sufren el mismo retardo de grupo,

$$\tau_g = \frac{N-1}{2\, f_s}. \tag{2}$$

La consigna fija, para cada filtro, la frecuencia de corte (o la frecuencia central y el ancho de banda, en el EB) y la atenuación mínima en la banda de rechazo, pero no el ancho de la banda de transición ni el rizado admitido en la banda de paso. Un filtro no queda definido sin esos dos datos, así que se eligieron al diseñar. La tabla 4 muestra ambas partes. En todos los casos se fijó $A_{\text{pass}} = 1\,\text{dB}$ (pico a pico) y se tomó A_{stop} igual a la pedida.

Tabla 4: Especificaciones de los filtros: lo que fija la consigna y los bordes elegidos. Las mismas bandas, en hertz, se usaron para las cinco f_s.

	Consigna	**Diseño**		
Filtro	**Bordes**	**Bordes [Hz]**	A_{pass}	A_{stop}
F1 PB	$f_c = 3\,600\,\text{Hz}$	$f_{\text{pass}} = 3500$, $f_{\text{stop}} = 3700$	1 dB	30 dB
F2 PA	$f_c = 35\,\text{Hz}$	$f_{\text{stop}} = 20$, $f_{\text{pass}} = 50$	1 dB	30 dB
F3 PBanda	35 Hz a 3 500 Hz	$f_{\text{stop1}} = 20$, $f_{\text{pass1}} = 50$ $f_{\text{pass2}} = 3400$, $f_{\text{stop2}} = 3600$	1 dB	30 dB
F4 EB	50 Hz, $BW = 15\,\text{Hz}$	$f_{\text{pass1}} = 35$, $f_{\text{stop1}} = 42{,}5$ $f_{\text{stop2}} = 57{,}5$, $f_{\text{pass2}} = 65$	1 dB	25 dB

Los bordes se ubicaron con un mismo criterio: la frecuencia de corte de la consigna queda en el centro de la banda de transición, que es donde un FIR de fase lineal con rizado chico cruza aproximadamente la mitad de la amplitud (−6 dB).

- En el PB, 3 600 Hz queda en el centro de la transición de 3 500 Hz a 3 700 Hz.
- En el PA y en el flanco inferior del PBanda, 35 Hz queda en el centro de 20 Hz a 50 Hz. En el flanco superior del PBanda, 3 500 Hz queda en el centro de 3 400 Hz a 3 600 Hz.
- En el EB, la banda de rechazo es exactamente la de la consigna, 50 Hz ± 7,5 Hz, con transiciones de 7,5 Hz a cada lado.

En la implementación previa (sección 3.3) la banda de paso del PA y del PBanda empezaba exactamente en 35 Hz, con la transición en 20 Hz a 35 Hz, y el flanco superior del PBanda usaba la misma transición que el PB. Esa transición de 15 Hz duplicaba el orden de esos filtros respecto de la actual.

3.2 Métodos de diseño

Un filtro FIR se puede diseñar por cuatro familias de métodos: por ventanas, por suavizado de la banda de transición, por muestreo en frecuencia y por métodos óptimos (los algoritmos de Parks-McClellan y de Remez). En este trabajo se usó el primero. Su punto de partida es la respuesta impulsional del filtro ideal, una función sinc de duración infinita. Si simplemente se la

trunca a N coeficientes, equivale a multiplicarla por una ventana rectangular, y en frecuencia eso convoluciona la respuesta ideal con la transformada de esa ventana: aparece el efecto Gibbs, un rizado que no disminuye aunque se agreguen coeficientes, y una atenuación de rechazo de sólo unos 21 dB. Multiplicar por una ventana que decae suavemente hacia los extremos elimina el corte abrupto y baja el rizado, a cambio de ensanchar la banda de transición. La elección de la ventana fija entonces ese compromiso.

En la implementación previa los cuatro filtros se diseñaron por el método de ventaneo con ventana de Kaiser: se trunca la respuesta impulsional ideal del filtro y se la multiplica por una ventana cuyo parámetro β regula el compromiso entre la atenuación en la banda de rechazo y el ancho de la transición. Una característica del método es que el rizado resulta igual en las dos bandas, $\delta = \min(\delta_p, \delta_s)$, con

$$\delta_p = \frac{10^{A_{\text{pass}}/20} - 1}{10^{A_{\text{pass}}/20} + 1} \approx 0{,}0575, \qquad \delta_s = 10^{-A_{\text{stop}}/20}. \tag{3}$$

Con $A_{\text{stop}} = 30\,\text{dB}$, $\delta_s \approx 0{,}0316$ es el más restrictivo, así que el rizado real de la banda de paso es de 0,55 dB, mejor que el 1 dB pedido. En el EB ($A_{\text{stop}} = 25\,\text{dB}$, $\delta_s \approx 0{,}0562$) los dos valores casi coinciden y el rizado de paso queda en 0,98 dB. La atenuación de diseño es $A = -20\log_{10}\delta$ y el parámetro de la ventana resulta $\beta \approx 2{,}12$ para 30 dB y $\beta \approx 1{,}33$ para 25 dB.

El orden M que requiere la ventana de Kaiser se estima con

$$M \approx \frac{A - 8}{2{,}285\,\Delta\omega}, \qquad \Delta\omega = \frac{2\pi\,\Delta f}{f_s}, \tag{4}$$

donde Δf es la banda de transición más angosta del filtro. La ecuación (4) se usa en la sección 3.3 para explicar la cantidad de coeficientes de cada filtro.

Como regla práctica, la teoría estima la longitud como $M \approx 4/BW$, con BW la banda de transición normalizada a f_s, medida entre el 99 % de la banda de paso y el 1 % de la de rechazo. Es la misma proporcionalidad inversa con la transición que contiene la ecuación (4), sólo que sin depender de la atenuación. Esos márgenes de 1 % corresponden a unos 40 dB, más exigentes que los 25 dB a 30 dB pedidos, por eso la regla sobrestima: para el PA de la implementación previa a 8 kS/s (transición de 15 Hz, $BW = 15/8000$) da unos 2 133 coeficientes, frente a los 822 de la ecuación (4).

Comparación con otras ventanas. Entre los métodos de ventaneo, las ventanas fijas tienen una atenuación en la banda de rechazo determinada por su forma y no ajustable: el diseñador sólo elige la cantidad de coeficientes. Con los valores de la tabla de ventanas de la cátedra (tabla 5), el error de aproximación pico es de -21 dB para la rectangular, -25 dB para la triangular, -44 dB para la de Hann, -53 dB para la de Hamming y -74 dB para la de Blackman, con anchos de transición equivalentes de $1{,}81\pi/N$, $2{,}37\pi/N$, $5{,}01\pi/N$, $6{,}27\pi/N$ y $9{,}19\pi/N$. Con las especificaciones de este trabajo (30 dB en el PB, el PA y el PBanda, y 25 dB en el EB) la rectangular no alcanza, y las de Hann, Hamming y Blackman cumplen con un exceso de atenuación que se paga en ancho de transición. Para $A = 30\,\text{dB}$, la ecuación (4) da $M \approx 9{,}6/\Delta\omega$, mientras que Hann necesitaría $\approx 15{,}7/\Delta\omega$, Hamming $\approx 19{,}7/\Delta\omega$ y Blackman $\approx 28{,}9/\Delta\omega$: 1,6, 2,0 y 3,0 veces más coeficientes, lo que es crítico cuando el EB a 48 kS/s ya necesita 7 604. El único caso en que una ventana fija coincide con Kaiser es el del EB: la triangular ($\approx 7{,}4/\Delta\omega$) equivale a Kaiser con $\beta = 1{,}33$, el valor que resulta para 25 dB, pero con margen nulo, y no existe una ventana fija equivalente para los 30 dB del resto. La ventana de Kaiser evita ese problema porque el parámetro β ajusta la atenuación al valor pedido, y además tiene una fórmula cerrada para estimar el orden a partir de A y $\Delta\omega$, lo que permite prever la cantidad de coeficientes y la carga de CPU antes de diseñar el filtro.

Tabla 5: Ventanas fijas frente a Kaiser (valores de la tabla de ventanas de la cátedra). La última columna es la cantidad de coeficientes respecto de Kaiser para $A = 30\,\text{dB}$.

Ventana	Lóbulo principal	Lóbulo lateral [dB]	Atenuación [dB]	β equiv.	Transición equiv.	Coef. vs. Kaiser
Rectangular	$4\pi/N$	-13	-21	0	$1{,}81\pi/N$	no cumple
Triangular	$8\pi/N$	-27	-25	1,33	$2{,}37\pi/N$	no cumple
Hann	$8\pi/N$	-32	-44	3,86	$5{,}01\pi/N$	1,6×
Hamming	$8\pi/N$	-43	-53	4,86	$6{,}27\pi/N$	2,0×
Blackman	$12\pi/N$	-58	-74	7,04	$9{,}19\pi/N$	3,0×
Kaiser ($A = 30\,\text{dB}$)	—	—	-30	2,12	$3{,}07\pi/N$	1,0×

Comparación con el diseño equiripple. La alternativa habitual, y la que `fdatool` propone por defecto, es el diseño equiripple por el algoritmo de Parks-McClellan. Ese método distribuye el error según la tolerancia de cada banda en lugar de imponer el mismo rizado en ambas, así que aprovecha la holgura de $A_{\text{pass}} = 1\,\text{dB}$ que la ventana de Kaiser desperdicia. Su orden se puede estimar con la fórmula de Kaiser para filtros óptimos,

$$M_{\text{eq}} \approx \frac{-20\log_{10}\sqrt{\delta_p\,\delta_s} - 13}{14{,}6\;\Delta f / f_s}, \tag{5}$$

pero es sólo una estimación (la misma que usa `firpmord` y, con ella, la opción *Minimum order* de `fdatool`) y puede quedar corta. El orden mínimo real se obtiene diseñando e iterando. Con los bordes de la tabla 4, a 8 kS/s el diseño equiripple necesita un orden de 48, 314 y 1 104 para el PB, el PA (y el PBanda) y el EB, frente a 62, 409 y 1 263 con la ventana de Kaiser: entre un 13 % y un 23 % menos, en la misma proporción para todas las f_s.

En la implementación previa se eligió la ventana de Kaiser por su robustez con filtros muy largos. El diseño por ventana es directo: se calcula la respuesta impulsional ideal y se la multiplica por la ventana, sin ningún proceso iterativo. Parks-McClellan, en cambio, es un algoritmo iterativo que, con órdenes de varios miles como los que se necesitan a las f_s altas (hasta 7 604 coeficientes en el EB a 48 kS/s), puede no converger o dar resultados numéricamente imprecisos. De hecho, `fdatool` devolvía coeficientes no finitos con esos órdenes, y para obtener los FIR directos equiripple de la sección 3.3 hubo que programar el algoritmo de Remez en un script propio. Ese diseño confirmó que el método no es la causa de fondo de la carga de CPU: el ahorro frente a Kaiser no alcanza para que el PA, el PBanda y el EB entren en tiempo real a las f_s altas.

En la implementación multitasa (sección 3.4) el argumento de la robustez desaparece: ningún bloque supera los 340 coeficientes, un orden en el que Parks-McClellan converge sin problemas (`firpm` de MATLAB). Por eso todos los bloques se diseñaron por equiripple, con el orden mínimo que cumple la especificación con los coeficientes ya cuantizados a q15.

Como el ADC puede muestrear a cinco velocidades, cada filtro tiene que tener la misma respuesta en frecuencia analógica cualquiera sea la velocidad activa, así que se diseña para cada f_s con las mismas bandas expresadas en hertz. En la implementación previa eso significó diseñar los 20 filtros, y al aumentar f_s la banda de transición ocupa una fracción menor del rango $[0,\ f_s/2]$ y el orden necesario crece (sección 3.3.2). En la implementación multitasa sólo se rediseñan para cada f_s los bloques que trabajan a esa tasa; los filtros angostos se diseñan una sola vez.

3.3 Implementación previa: un FIR directo por velocidad

La primera implementación calculaba cada uno de los 20 filtros como un FIR directo (ecuación (1)) a su propia f_s, con `arm_fir_q15()`. Los filtros se diseñaron en `fdatool` por ventana

de Kaiser, con los bordes anteriores del PA y del PBanda (transición de 20 Hz a 35 Hz). Las figuras 2 a 5 muestran los diseñados para 8 kS/s.

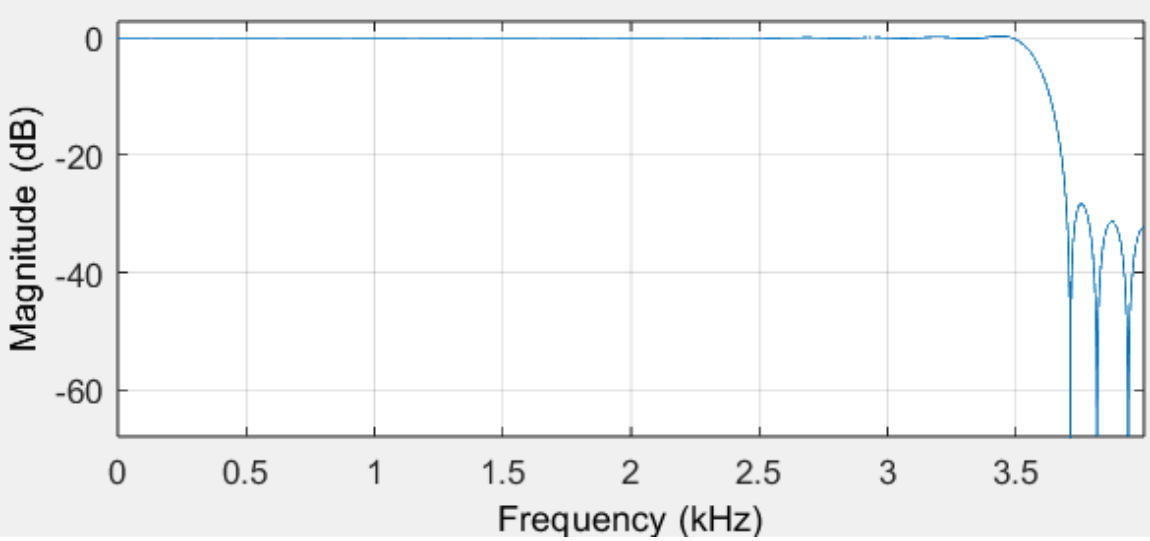


Figura 2: fdatool. Respuesta en magnitud del filtro pasa bajos para $f_s = 8\,\text{kS/s}$ (implementación previa).

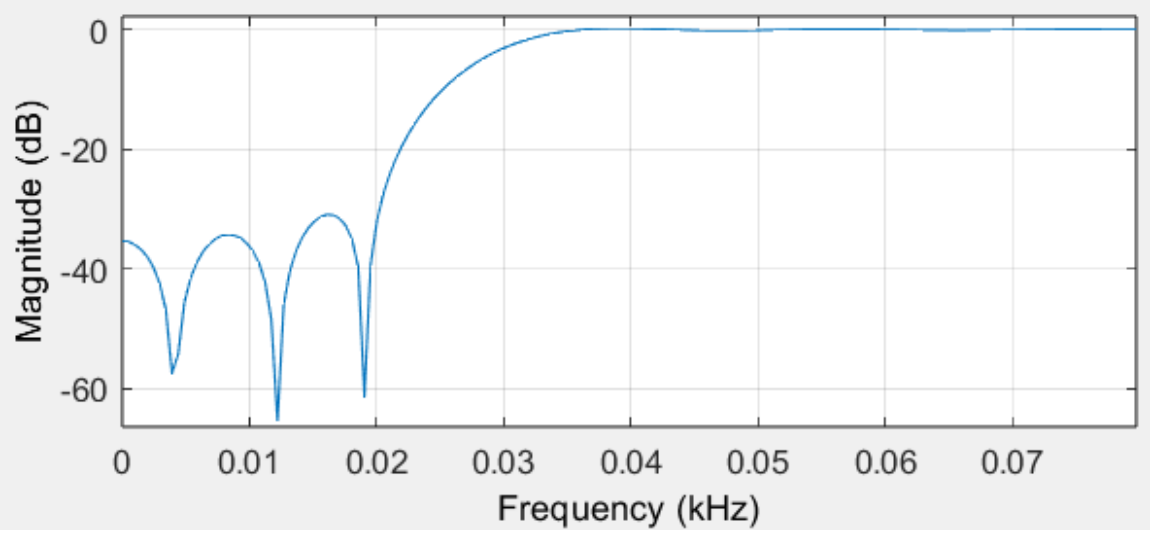


Figura 3: fdatool. Respuesta en magnitud del filtro pasa altos para $f_s = 8\,\text{kS/s}$ (implementación previa).

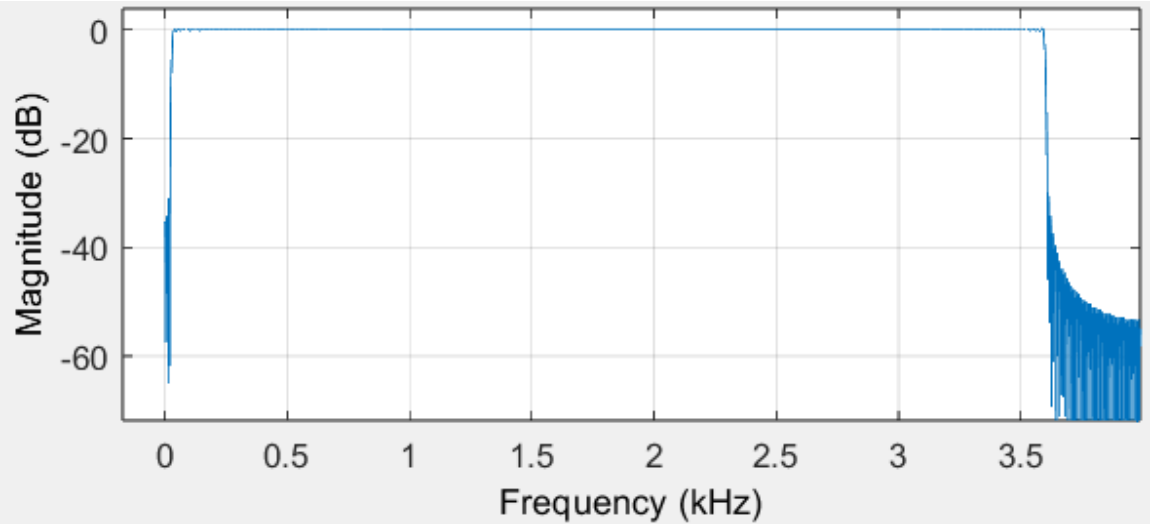


Figura 4: fdatool. Respuesta en magnitud del filtro pasa banda para $f_s = 8\,\text{kS/s}$ (implementación previa).

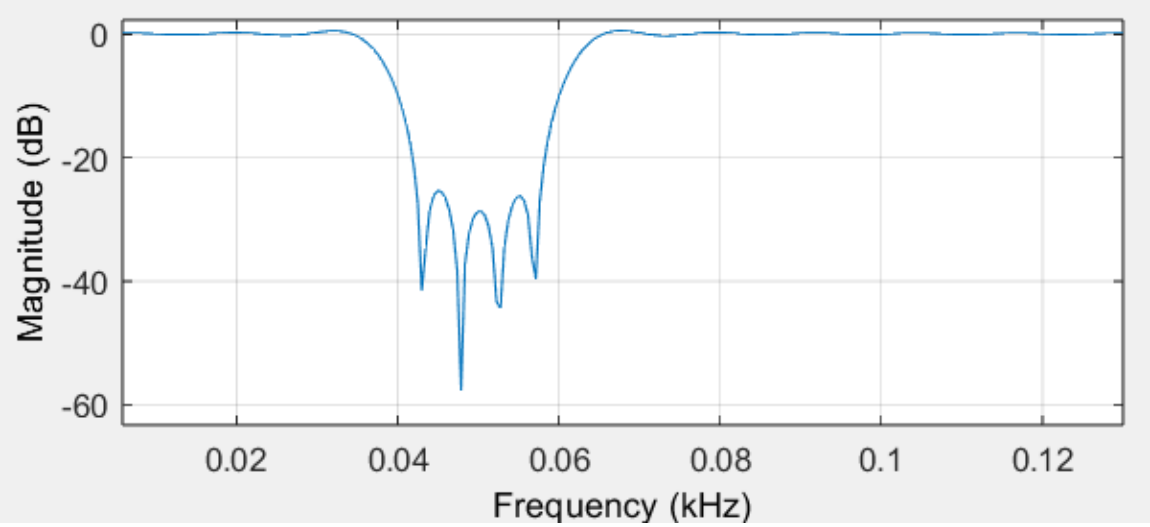


Figura 5: fdatool. Respuesta en magnitud del filtro elimina banda para $f_s = 8\,\text{kS/s}$ (implementación previa).

3.3.1 Generación de los coeficientes

Los coeficientes de cada filtro se exportaron desde `fdatool` al *workspace* de MATLAB (*File* → *Export*, como *Coefficients*) con el nombre `<tipo>_<fs>`, por ejemplo `PB_8k` o `EB_48k`. El script `generar_filtros.m` toma las 20 variables y, para cada una:

1. Si algún coeficiente alcanza $|b_k| \geq 1$ (que no se puede representar en q15), escala todo el filtro para que el mayor quede en 32767/32768. La variación de ganancia es menor a 0,01 dB.
2. Cuantiza a q15: $b_k^{q15} = \text{round}(2^{15}\, b_k)$.
3. Invierte el orden de los coeficientes, porque CMSIS-DSP los espera como $\{b_{N-1}, \ldots, b_1, b_0\}$. En un filtro simétrico la inversión no cambia nada, pero el script no depende de eso.
4. Si N es impar agrega un coeficiente nulo, porque `arm_fir_init_q15()` exige un número par de coeficientes mayor o igual a 4. Como el arreglo está invertido, ese cero queda como nuevo b_0: no modifica la respuesta en magnitud y sólo agrega una muestra de retardo.

5. Escribe los arreglos `static const q15_t` en el archivo de cabecera, junto con dos tablas indexadas por [f_s][tipo]: `coef_tabla` (punteros a los coeficientes) y `taps_tabla` (cantidad de coeficientes). También define `MAX_TAPS`, el largo del filtro más largo.

El script además imprime una estimación de la carga de CPU de cada filtro, que se analiza en la sección 4.8. El archivo generado (`FiltrosTP2.h`) se incorporó al proyecto como `source/filtros.h`.

3.3.2 Cantidad de coeficientes y memoria

Tabla 6: Cantidad de coeficientes de cada filtro (después del relleno a un número par), tomada de `taps_tabla` en `source/filtros.h`.

f_s	PB	PA	PBanda	EB	Total
8 000 Hz	64	822	822	1 270	2 978
16 000 Hz	124	1 642	1 640	2 536	5 942
22 000 Hz	170	2 256	2 254	3 486	8 166
44 000 Hz	340	4 508	4 508	6 970	16 326
48 000 Hz	370	4 918	4 916	7 604	17 808
Total					51 220

Se observan dos cosas:

- La cantidad de coeficientes crece casi en proporción a f_s (por ejemplo, el EB pasa de 1 270 coeficientes a 8 kS/s a 7 604 a 48 kS/s, unas 6 veces, igual que la relación entre las frecuencias de muestreo). Es lo esperable al mantener fijas las bandas en hertz: el ancho de transición normalizado $\Delta f / f_s$ se achica en la misma proporción y el orden de un FIR es inversamente proporcional a ese ancho.
- El PB es mucho más corto que los otros tres filtros, aunque su atenuación es la misma que la del PA y el PBanda. La diferencia está en el ancho de la banda de transición, como se explica a continuación.

La ecuación (4) explica esa diferencia. La transición más angosta de cada filtro, con los bordes de la implementación previa, es de 200 Hz en el PB, 15 Hz en el PA y el PBanda (de 20 Hz a 35 Hz) y 7,5 Hz en el EB. La tabla 7 compara el orden que predice la fórmula con la cantidad de coeficientes implementada, que es $M + 1$ más el eventual cero de relleno.

Tabla 7: Orden estimado con la ecuación (4) y cantidad de coeficientes implementada, a la menor y a la mayor f_s.

			8 kS/s		48 kS/s	
Filtro	Δf	A	M **estimado**	N **impl.**	M **estimado**	N **impl.**
PB	200 Hz	30 dB	62	64	368	370
PA	15 Hz	30 dB	818	822	4 904	4 918
PBanda	15 Hz	30 dB	818	822	4 904	4 916
EB	7,5 Hz	25 dB	1 264	1 270	7 579	7 604

La estimación coincide con lo implementado con un error menor al 1 % en todos los casos, lo

que confirma que los filtros de la implementación previa tenían esos bordes y que se usaron las mismas bandas en hertz para todas las f_s.

La explicación de fondo está en dónde caen los bordes. El PB tiene su transición en 3 500 Hz a 3 700 Hz y puede ser de 200 Hz. En cambio, el PA y el flanco inferior del PBanda pasan de rechazo a paso entre 20 Hz y 35 Hz, y el EB tiene que entrar y salir de una banda de rechazo de 15 Hz con transiciones de 7,5 Hz. Según la ecuación (4), M es proporcional a $f_s/\Delta f$: una transición de unos pocos hertz con f_s de miles de hertz requiere miles de coeficientes. Ese es el origen de la carga de CPU que se analiza en la sección 4.8.

A 8 kS/s el PB tiene además una particularidad: la frecuencia de Nyquist es 4 000 Hz y la banda de rechazo empieza en 3 700 Hz, así que sólo atenúa los últimos 300 Hz del espectro representable. A esa f_s, su efecto sobre una señal de prueba es pequeño y se observa recién por encima de 3,5 kHz.

Los 51 220 coeficientes ocupan 102 440 bytes (unos 100 KiB) de memoria flash, una fracción chica de los 2 MB del MCXN947. En RAM sólo hace falta un buffer de estado de `MAX_TAPS`$+1 = 7\,605$ muestras (15 210 bytes), compartido por todos los filtros.

Con la ecuación (2), el retardo de grupo resulta casi igual para todas las f_s, porque N crece en la misma proporción que f_s. El filtro más largo, el EB, retrasaba la señal unos $1\,269/(2\cdot 8\,\text{kHz}) \approx 79$ ms.

3.3.3 Diseño equiripple de orden mínimo

Para separar el efecto del método de diseño del de la estructura, los 20 filtros directos se rediseñaron por equiripple con el script `disenar_PB_PA.m`, ya con los bordes de la tabla 4. El script implementa el algoritmo de Remez y, para cada filtro, busca el menor orden que cumple la especificación con los coeficientes ya cuantizados a q15. La tabla 8 muestra el resultado. Parte de la reducción respecto de la tabla 6 se debe a los bordes (la transición del PA y del PBanda pasó de 15 Hz a 30 Hz); con los mismos bordes, el ahorro del método equiripple frente a Kaiser es de un 13 % a 23 % (sección 3.2).

Tabla 8: Cantidad de coeficientes de los FIR directos equiripple de orden mínimo (`disenar_PB_PA.m`), después del relleno a un número par.

f_s	PB	PA	PBanda	EB	Total
8 000 Hz	50	318	316	1 108	1 792
16 000 Hz	94	632	634	2 216	3 576
22 000 Hz	128	870	872	3 048	4 918
44 000 Hz	256	1 744	1 746	6 124	9 870
48 000 Hz	278	1 902	1 904	6 680	10 764
Total					30 920

3.3.4 Por qué el FIR directo no alcanza

Aun con el orden mínimo, el FIR directo tiene tres problemas que no dependen del método de diseño:

- **Carga de CPU.** Como $N \propto f_s/\Delta f$ y el filtro se calcula f_s veces por segundo, la carga crece con f_s^2. Con la estimación de la sección 4.8, el EB a 48 kS/s necesitaría más de 10 veces el período de muestreo, y a partir de 44 kS/s sólo el PB entra en tiempo real.

- **Precisión de los coeficientes.** En un filtro de banda angosta con f_s alta, la respuesta impulsional se reparte entre miles de coeficientes muy chicos. En el EB a 48 kS/s, el 38 % de los 6 680 coeficientes vale 2 LSB o menos en q15: la cuantización deja muy pocos bits efectivos justamente en la parte del filtro que define la banda angosta.
- **Memoria.** Los coeficientes ocupan decenas de kilobytes y el buffer de estado crece con el filtro más largo.

Los tres tienen el mismo origen: se resuelve a f_s un detalle de pocos hertz. Eso es lo que corrige la implementación multitasa.

3.4 Implementación multitasa

3.4.1 Idea

Las bandas que exigen transiciones angostas están todas por debajo de 65 Hz: la transición de 20 Hz a 50 Hz del PA y del flanco inferior del PBanda, y la banda de rechazo de 42,5 Hz a 57,5 Hz del EB con sus transiciones de 7,5 Hz. Para resolver esos detalles no hace falta trabajar a 48 kS/s: alcanza con una tasa algunas veces mayor que 65 Hz. Si la señal se lleva a una tasa baja f_L, el mismo Δf ocupa una fracción f_s/f_L veces mayor del rango normalizado y, por la ecuación (4), el orden baja en esa misma proporción.

Se eligió $f_L = 1\,\text{kHz}$ porque las cinco velocidades son múltiplos enteros de ella: el factor de diezmado es $M = f_s/f_L = 8$, 16, 22, 44 y 48. Con un factor entero el cambio de tasa es exacto (se toma una de cada M muestras) y un único juego de filtros a 1 kHz sirve para las cinco f_s. Las f_s reales difieren de las nominales en menos de un 0,04 % (ecuación (15)); M sigue siendo entero y todas las frecuencias del filtro se corren en esa misma proporción, lo que es despreciable.

3.4.2 Estructura

La figura 6 muestra la estructura. El *camino angosto* calcula, a partir de $x[n]$, una señal $v[n]$ que contiene sólo la parte de baja frecuencia que el filtro tiene que tratar:

$$u[k] = \sum_{j=0}^{N_D-1} h_D[j]\, x[kM - j] \qquad \text{(filtro antialias y diezmado)}, \tag{6}$$

$$w[k] = \sum_{i=0}^{N_G-1} g[i]\, u[k - i] \qquad \text{(filtro angosto a } f_L\text{)}, \tag{7}$$

$$v[n] = M \sum_{k} w[k]\, h_D[n - kM] \qquad \text{(interpolación)}. \tag{8}$$

El pasa altos y el elimina banda salen por complemento, restando el camino angosto de la entrada retardada:

$$y[n] = x[n - D] - v[n], \qquad D = (N_D - 1) + M\,\frac{N_G - 1}{2}. \tag{9}$$

Para el PA, G es un pasa bajos (pasa 0 Hz a 20 Hz y rechaza desde 50 Hz), y $1 -$ pasa bajos es un pasa altos. Para el EB, G es un pasa banda de 42,5 Hz a 57,5 Hz, y $1 -$ pasa banda es un elimina banda. El PBanda es el PA seguido de un pasa bajos directo a f_s (LPB, 3 400 Hz a 3 600 Hz). El PB queda como FIR directo a f_s: su transición de 200 Hz no es angosta y el filtro ya era corto (tabla 8).

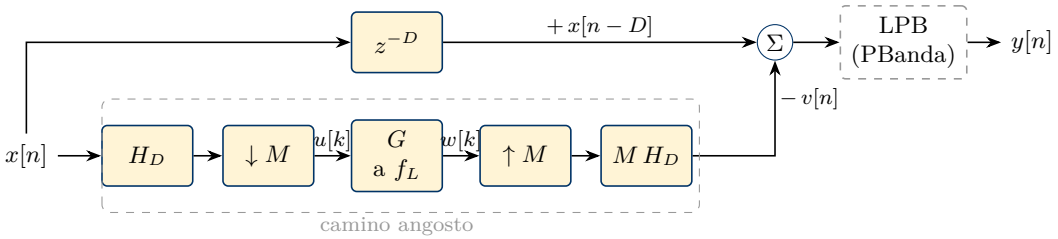


Figura 6: Estructura multitasa del PA y del EB ($G = G_{\text{PA}}$ o G_{EB}). El PBanda agrega a la salida el pasa bajos LPB; el PB es un FIR directo a f_s.

Tres decisiones hacen que el complemento funcione:

- Todos los filtros del camino son de fase lineal y de longitud impar, así que su retardo es un número entero de muestras y D es exacto. Si $v[n]$ tuviera un retardo distinto del de $x[n-D]$, la resta no anularía la banda.
- El mismo h_D sirve de diezmador y de interpolador (este último con ganancia M, que compensa los $M-1$ ceros que inserta la interpolación): los dos tienen que dejar pasar 0 Hz a 65 Hz y rechazar lo que, al cambiar de tasa, cae encima de esa banda, que empieza en $f_L - 65\,\text{Hz} = 935\,\text{Hz}$.
- Los arreglos con el formato de CMSIS-DSP se completan a un número par de coeficientes con el cero al *principio* del arreglo, que en el orden invertido de CMSIS es el final de la respuesta impulsional. La implementación previa lo agregaba al final del arreglo, lo que sumaba una muestra de retardo. En un FIR directo esa muestra no importa; en G se multiplicaría por M y rompería el complemento.

Si por un momento se ignoran los efectos del cambio de tasa, la estructura equivale a un filtro LTI con respuesta en amplitud

$$A(f) = 1 - A_D(f)^2\, A_G(f), \tag{10}$$

donde A_D y A_G son las respuestas de fase cero de h_D y de g (esta última periódica, con período f_L). La ecuación muestra cómo se reparte el error: en la banda de rechazo del PA, donde $A_G \approx 1$, el error es la suma de los rizados de G y de las dos pasadas por H_D; en su banda de paso, donde $A_G \approx 0$, el rizado del PA es el rechazo de G.

3.4.3 Aliasing e imágenes

Un sistema con cambio de tasa no es LTI: una senoidal de frecuencia f a la entrada produce, además de la salida en f que describe la ecuación (10), componentes en $f + kf_L$ ($k \neq 0$) de amplitud

$$|H_D(f)|\;|G(f)|\;|H_D(f + kf_L)|. \tag{11}$$

Se exigió que la suma en potencia de todas esas componentes quedara en -40 dB o menos, 10 dB por debajo de la banda de rechazo más exigente.

El caso más desfavorable no está donde H_D rechaza, sino en su banda de transición (65 Hz a 935 Hz). Ahí H_D deja pasar parte de la señal, que al diezmar se superpone con la banda de rechazo de G, y el interpolador la vuelve a sacar en $f_L - f$. Su nivel es aproximadamente el rechazo de G multiplicado por $|H_D(f_L/2)|^2 \approx 1/4$. En un primer diseño, con el rechazo de G_{EB} en -26 dB, esas componentes llegaban a $-38{,}8$ dB. Por eso el desvío en la banda de rechazo de los filtros G se limitó también a $4 \cdot 10^{-40/20}$, lo que alargó G_{EB} de 143 a 187 coeficientes.

3.4.4 Relación con la FFT

La palabra *diezmado* aparece también en la FFT (algoritmos de diezmado en el tiempo y en la frecuencia), y la relación va más allá del nombre: las dos técnicas se apoyan en la misma separación de una secuencia en fases y en el mismo efecto de esa separación sobre el espectro. Esa relación explica por qué la estructura multitasa es eficiente, por qué necesita el filtro H_D y cómo se verificaron los filtros.

Separar en fases. La FFT de diezmado en el tiempo separa la secuencia en sus muestras pares e impares y arma la DFT de N puntos con dos DFT de $N/2$:

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r]\, W_{N/2}^{rk}}_{P[k]} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x[2r+1]\, W_{N/2}^{rk}}_{I[k]}, \qquad W_N = e^{-j2\pi/N}. \tag{12}$$

Como $W_N^{N/2} = -1$, cada par de salidas sale de una *mariposa*: $X[k] = P[k] + W_N^k I[k]$ y $X[k + N/2] = P[k] - W_N^k I[k]$, para $0 \le k < N/2$. Repitiendo la separación se llega a $\log_2 N$ etapas y a unas $(N/2)\log_2 N$ multiplicaciones en lugar de N^2. Con M fases en lugar de 2 (FFT de base M) las secuencias son $x[rM + p]$, con $p = 0, \ldots, M-1$: son las *componentes polifásicas* de x.

La estructura multitasa usa la misma separación, aplicada al filtro h_D. Sus componentes polifásicas son $e_p[m] = h_D[mM + p]$, y con ellas las ecuaciones (6) y (8) quedan

$$u[k] = \sum_{p=0}^{M-1} \sum_m e_p[m]\; x\big[(k-m)M - p\big], \qquad v[kM+p] = M \sum_m e_p[m]\; w[k-m]. \tag{13}$$

En el diezmador, cada fase del filtro trabaja sobre una fase de la entrada, como cada DFT de $N/2$ puntos trabaja sobre una mitad de la secuencia. En el interpolador, la fase p de la salida sale sólo de la fase p del filtro: es la fórmula de la sección 4.6.3, que saltea las multiplicaciones por los ceros insertados. El ahorro tiene el mismo origen en los dos casos: no hacer cuentas innecesarias. En la FFT, cada DFT más chica se calcula una vez y se reutiliza para dos salidas; en el diezmador, no se calculan las $M-1$ de cada M salidas que se descartan; en el interpolador, no se multiplica por los ceros.

Diezmar en el tiempo es plegar en frecuencia. Si se suman las dos salidas de la mariposa, queda

$$P[k] = \tfrac{1}{2}\big(X[k] + X[k+N/2]\big) \qquad \text{y, con } M \text{ fases,} \qquad X_0[k] = \frac{1}{M} \sum_{q=0}^{M-1} X\left[k + q\,\frac{N}{M}\right], \tag{14}$$

donde X_0 es la DFT de $x[rM]$. La DFT de las muestras que se conservan es el espectro original *plegado*: cada bin suma los M bins, separados N/M, que al bajar la tasa caen en el mismo lugar. Es el mismo aliasing de la ecuación (11), escrito con la DFT: diezmar por M en el tiempo superpone las frecuencias $f, f + f_L, \ldots, f + (M-1)f_L$.

La diferencia está en lo que se hace después. La FFT conserva *todas* las fases, y la mariposa deshace el plegado: con $P[k]$ e $I[k]$ recupera $X[k]$ y $X[k+N/2]$ por separado, así que el cálculo es exacto y no necesita ningún filtro. El diezmado de la señal conserva una sola fase y descarta las otras $M-1$, así que el plegado ya no se puede deshacer: lo que haya en $f + qf_L$ queda mezclado con lo que hay en f. Por eso hace falta H_D *antes* de diezmar: para que en esas frecuencias no haya nada que plegar.

La FFT en la generación de los filtros. Los scripts que generan los coeficientes usan la FFT en tres lugares:

- **Para medir cada bloque.** La respuesta en frecuencia de un FIR es la DFT de sus coeficientes completados con ceros: `abs(fft(h, nfft))` da $|H|$ en `nfft` frecuencias equiespaciadas. Con `nfft` $\geq 32N$ hay unos 32 puntos por lóbulo, y sobre esa grilla se decide si el bloque cumple en q15.
- **Para medir el aliasing.** Para evaluar la ecuación (11), `disenar_multitasa.m` usa una FFT de $N_{\text{FFT}} = MK$ puntos, de modo que f_L equivale exactamente a K bins. Así los bins $i, i+K, \ldots, i+(M-1)K$ son las M frecuencias que se pliegan entre sí, y la suma de la ecuación (14) se calcula reacomodando el vector en una matriz de $K \times M$ y sumando por filas. Es la misma condición que pide una FFT de base M: que la cantidad de puntos sea múltiplo de M.
- **Para diseñar los FIR directos largos.** El algoritmo de Remez de `disenar_PB_PA.m` evalúa en cada iteración la respuesta $A(\omega) = \sum_k a_k \cos(k\omega)$ en una grilla de $16L$ frecuencias ($L \approx N/2$). Calcularla directamente cuesta unos $16L^2$ productos por iteración, más de 10^8 para el EB a 48 kS/s; con una FFT del vector a_k, del orden de $32L \log_2(32L)$. Eso es lo que permitió llegar a órdenes de miles, donde `fdatool` fallaba.

Además, de las cuatro familias de métodos de diseño (sección 3.2), la de muestreo en frecuencia obtiene los coeficientes directamente como la DFT inversa de N muestras de la respuesta deseada, es decir, con una IFFT. La tabla 9 resume la comparación.

Tabla 9: El diezmado en la FFT y en la estructura multitasa.

	FFT (diezmado en el tiempo)	**Diezmador e interpolador**
Qué se separa en fases	La secuencia, $x[rM+p]$	El filtro, $e_p[m] = h_D[mM+p]$, y con él la entrada
Qué se hace con las fases	Se conservan todas y se recombinan con los factores W_N^{pk}	Se conserva una muestra de cada M (diezmado); en cada muestra se calcula una sola fase (interpolación)
Efecto en frecuencia	Cada fase tiene el espectro plegado (ecuación (14)); la mariposa lo deshace	El plegado es el aliasing y no se puede deshacer
¿Necesita filtro?	No: el cálculo es exacto	Sí: H_D antes de diezmar y después de interpolar
De dónde sale el ahorro	Reutilizar DFT más chicas: de N^2 a $(N/2)\log_2 N$ productos	No calcular salidas descartadas ni productos por cero: N_D productos cada M muestras, o L por muestra con el diezmador repartido

3.4.5 Reparto del error

Las especificaciones de la tabla 4 son del filtro completo, así que hay que repartirlas entre los bloques. Con $\delta_p = 0{,}0575$ (1 dB pico a pico), $\delta_s = 10^{-A_{\text{stop}}/20}$ y un margen del 85 % para absorber la cuantización:

- H_D: desvío de 0,002 en la banda de paso y 60 dB de rechazo. Su rizado entra dos veces en la ecuación (10).
- G_{PA}: en su banda de paso (la de rechazo del PA), el desvío tiene que dejar lugar a las dos pasadas por H_D, $0{,}85\,(\delta_s - 2 \cdot 0{,}002) = 0{,}0235$. En su banda de rechazo (la de paso del PA) se usa la mitad del rizado de 1 dB, porque el PBanda usa el PA en cascada con el LPB: $\delta_{pc} = (10^{1/40} - 1)/(10^{1/40} + 1) = 0{,}0288$, que con el margen da 0,0245.

- G_{EB}: $0{,}85\,(\delta_s - 2\cdot 0{,}002) = 0{,}0444$ en su banda de paso (la de rechazo del EB, con $A_{\mathrm{stop}} = 25\,\mathrm{dB}$) y 0,034 en las de rechazo, por el límite de las componentes espurias.
- LPB: el rizado que deja libre el PA dentro de 1 dB (0,0280) y un rechazo de 0,0262, que contempla la ganancia del PA en su banda de paso.
- PB: directamente δ_p y δ_s.

Tabla 10: Bloques de la implementación multitasa: bandas, desvíos admitidos y cantidad de coeficientes (orden mínimo que cumple en q15). Para los bloques a f_s se indica el valor a 8, 16, 22, 44 y 48 kS/s.

Bloque	Tasa	Bandas [Hz]	δ paso	δ rechazo	Coeficientes
G_{PA}	f_L	paso 0–20; rechazo 50–500	0,023 5	0,024 5	49
G_{EB}	f_L	paso 42,5–57,5; rechazo 0–35 y 65–500	0,044 4	0,034 0	187
H_D	f_s	paso 0–65; rechazo 935–$f_s/2$	0,002	0,001	31, 59, 83, 165, 181
LPB	f_s	paso 0–3400; rechazo 3600–$f_s/2$	0,028 0	0,026 2	57, 113, 155, 309, 337
PB	f_s	paso 0–3500; rechazo 3700–$f_s/2$	0,057 5	0,031 6	49, 93, 129, 255, 279

3.4.6 Diseño y verificación

El script `disenar_multitasa.m` diseña cada bloque con `firpm` (Parks-McClellan): parte del orden que estima `firpmord` (un 15 % por debajo, porque la fórmula puede equivocarse en los dos sentidos) y lo sube de a 2 hasta que los coeficientes, *ya redondeados a q15*, cumplen los desvíos de la tabla 10. Después verifica cada uno de los 20 filtros compuestos contra la especificación original:

1. Calcula la respuesta exacta de la parte LTI (la convolución de h_D, g con $M-1$ ceros entre coeficientes y otra vez h_D, y el complemento) y mide el rizado, la atenuación y la ganancia máxima en todo el rango $[0,\ f_s/2]$.
2. Calcula con la ecuación (11) el nivel total de aliasing e imágenes para cada frecuencia de entrada.
3. Simula la cadena en el tiempo con dos modelos, uno vectorizado y otro muestra a muestra, que es el que se tradujo a C. Comprueba que coinciden (diferencia máxima de $6\cdot 10^{-17}$) y que la amplitud de tonos de prueba medida en régimen coincide con la respuesta calculada (diferencia máxima de $5\cdot 10^{-7}$).

La tabla 11 resume el resultado: los 20 filtros cumplen.

Tabla 11: Filtros compuestos con coeficientes q15, peor caso entre las cinco f_s: rizado pico a pico en la banda de paso, atenuación mínima en la de rechazo, nivel total de aliasing e imágenes y retardo de grupo.

Filtro	Rizado	Atenuación	Pedida	Espurias	Retardo
PB	0,99 dB	30,06 dB	30 dB	—	3,0 ms
PA	0,38 dB	32,28 dB	30 dB	−45,6 dB	27,8 ms
PBanda	0,79 dB	31,74 dB	30 dB	−45,4 dB	31,3 ms
EB	0,59 dB	26,80 dB	25 dB	−41,8 dB	96,8 ms

Las figuras 7 a 10 muestran la respuesta en frecuencia de cada filtro compuesto, calculada con los coeficientes q15, a las cinco f_s superpuestas y con eje de frecuencia logarítmico. La línea de trazos es la atenuación pedida y la de puntos, el límite inferior de la banda de paso; cada curva llega hasta su $f_s/2$. Las cinco curvas casi coinciden: la respuesta en hertz no depende de la velocidad. En el PA, el PBanda y el EB eso se cumple por construcción, porque la parte angosta se calcula siempre a 1 kHz con los mismos filtros G. Las figuras muestran la parte LTI de la respuesta; el nivel de las componentes espurias del cambio de tasa está en la tabla 11.

La carga de CPU que indican las leyendas es la de la muestra de diezmado con el diezmador calculado de una sola vez. El firmware lo reparte entre las M muestras, así que su carga es la de la tabla 17: por ejemplo, 63 % y no 92 % en el PBanda a 48 kS/s.

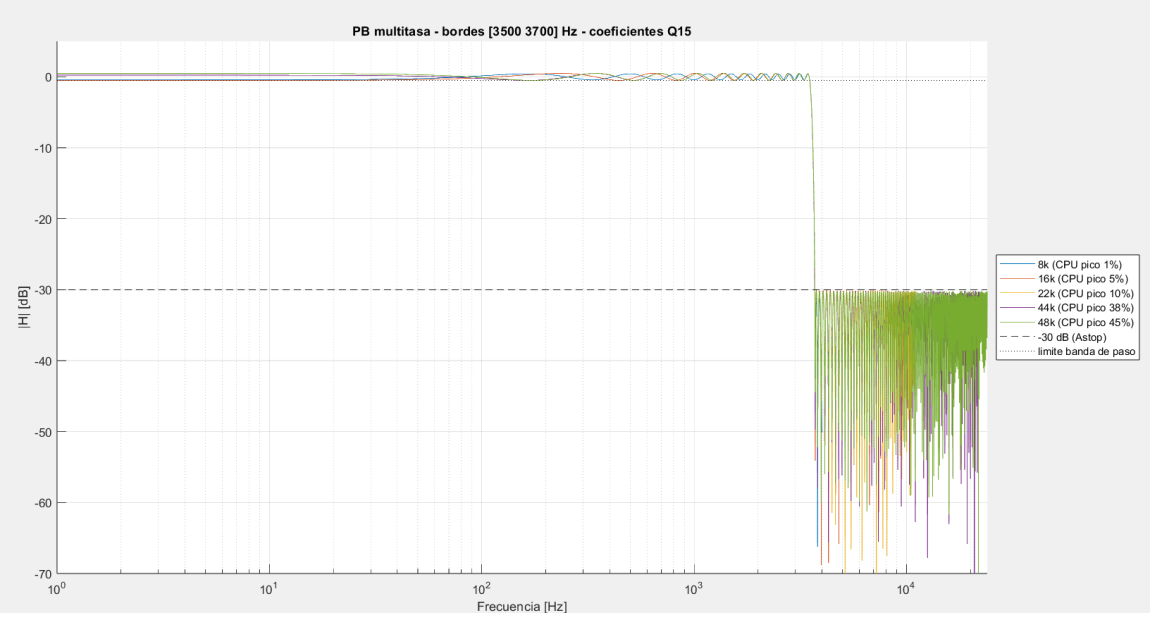


Figura 7: MATLAB. Respuesta del pasa bajos a las cinco f_s, con coeficientes q15. Es un FIR directo; el título dice *multitasa* porque el script grafica los cuatro filtros con la misma función.

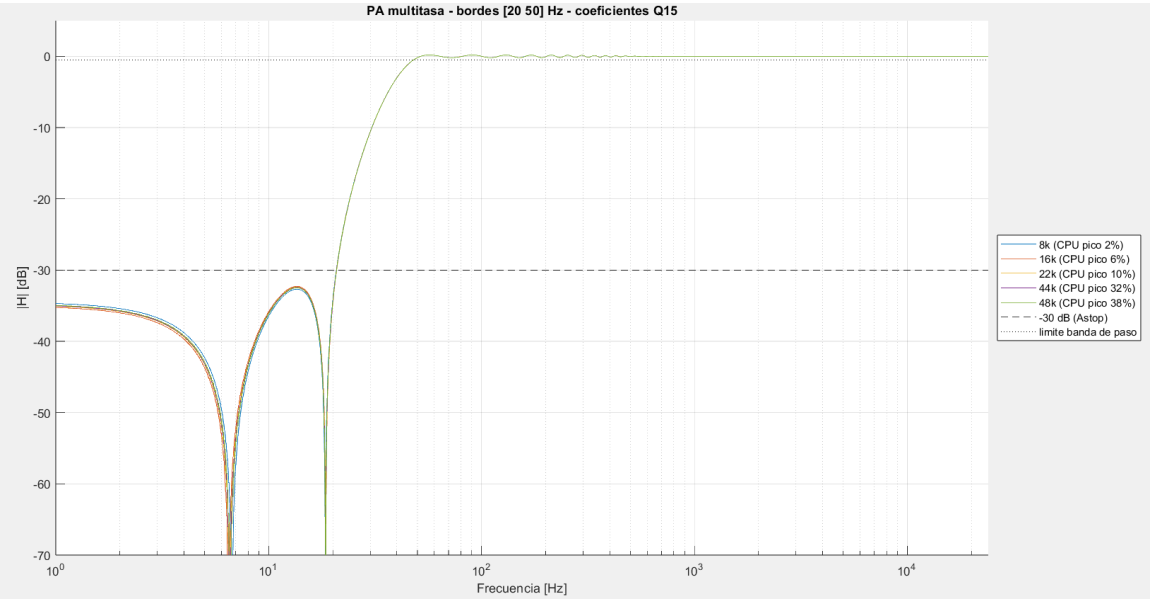


Figura 8: MATLAB. Respuesta del pasa altos multitasa a las cinco f_s: rechazo por debajo de -30 dB hasta 20 Hz y banda de paso desde 50 Hz.

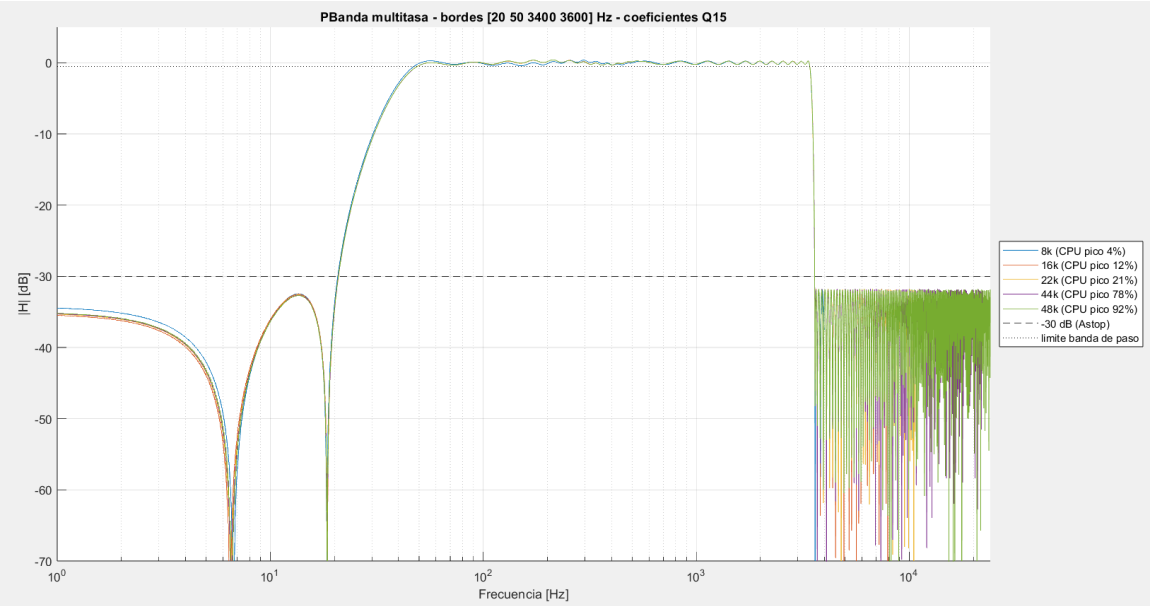


Figura 9: MATLAB. Respuesta del pasa banda multitasa (PA seguido del LPB) a las cinco f_s.

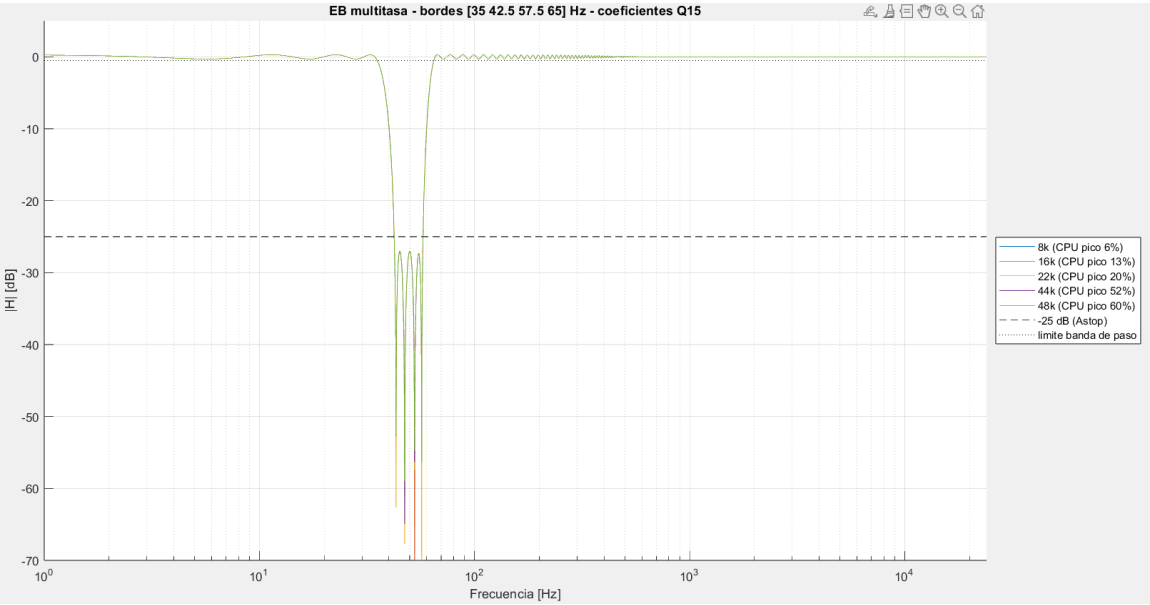


Figura 10: MATLAB. Respuesta del elimina banda multitasa a las cinco f_s: rechazo por debajo de $-25\,\mathrm{dB}$ entre 42,5 Hz y 57,5 Hz.

3.5 De MATLAB al firmware

`disenar_multitasa.m` escribe directamente el archivo de cabecera `source/filtros_multitasa.h`, sin pasar por `fdatool`. Además de los coeficientes en q15, $b_k^{q15} = \mathrm{round}(2^{15}\, b_k)$, el archivo trae todo lo que el firmware necesita para armar la estructura:

- `G_PA` y `G_EB`, compartidos por las cinco f_s, y `PB_<fs>` y `LPB_<fs>`, en el formato de CMSIS-DSP (orden invertido y cantidad par de coeficientes, con el cero al principio del arreglo).
- `HD_<fs>` en orden natural, completado con ceros hasta un múltiplo de M (`HD_PAD`), para

que el interpolador polifásico (sección 4.6.3) pueda recorrer cada fase sin controlar los límites.

- Las tablas `m_tabla` (M), `d_pa_tabla` y `d_eb_tabla` (D de la ecuación (9)), las tablas de punteros y de cantidades indexadas por f_s, y las constantes que dimensionan los buffers (`LONG_HIST_X`, `MAX_HIST_W` y `MAX_TAPS_FS`).

El mismo script escribe `resumen_multitasa.csv`, con la carga de CPU, el retardo y el resultado de la verificación de cada filtro, y grafica las respuestas compuestas. El archivo de cabecera completo, con los valores de todos los coeficientes, está en el Anexo **??**.

3.6 Comparación con la implementación previa

La tabla 12 resume las diferencias. Las cargas de CPU son estimaciones con $c = 5$ ciclos por coeficiente (sección 4.8); el peor caso de la multitasa además se midió (sección 4.8.1).

Tabla 12: Implementación previa (un FIR directo por f_s) frente a la implementación multitasa.

	Previa: FIR directo	**Multitasa**
Estructura	20 FIR, cada uno calculado a su f_s	PB directo; PA y EB por complemento de un camino a 1 kHz; PBanda = PA seguido de LPB
Diseño	`fdatool`, ventana de Kaiser, exportado con `generar_filtros.m`	`firpm` (equiripple), orden mínimo verificado en q15, con `disenar_multitasa.m`
Bordes del PA y del PBanda	Banda de paso desde 35 Hz (transición de 15 Hz)	f_c = 35 Hz en el centro de 20 Hz a 50 Hz
Filtros que cambian con f_s	Los 20	H_D, PB y LPB; G_{PA} y G_{EB} son los mismos
Coeficientes en flash	51 220 (100 KiB); 30 920 con equiripple	2 576 (5 KiB)
RAM de estado	7 605 muestras (15 KiB)	unos 10 KiB, casi todo la historia de x para $x[n - D]$
Peor carga estimada	EB a 48 kS/s: 1 217 % (Kaiser), 1 069 % (equiripple)	PBanda a 48 kS/s: 63 % (medido: 65 %)
Combinaciones en tiempo real (estimado)	10 de 20 (Kaiser), 12 de 20 (equiripple)	20 de 20
Retardo del PA	51 ms (Kaiser), 20 ms (equiripple)	28 ms
Retardo del EB	79 ms (Kaiser), 70 ms (equiripple)	97 ms
Coeficientes q15 de 2 LSB o menos	38 % en el EB a 48 kS/s (equiripple)	1,2 % en G_{EB}
Firmware	`arm_fir_q15()` sobre 20 instancias	`arm_fir_q15()` para PB, LPB y G; diezmador repartido e interpolador polifásico propios

La tabla 13 da la cantidad de coeficientes de cada uno de los 20 filtros finales, sumando los bloques que usa: el PB, sólo el suyo; el PA, H_D y G_{PA}; el PBanda, además, el LPB; y el EB, H_D y G_{EB}. H_D se cuenta una vez aunque se usa dos veces (como diezmador y como interpolador), y no se cuentan los ceros de relleno. Entre paréntesis figura la cantidad del FIR directo equiripple de la tabla 8, con las mismas especificaciones.

Tabla 13: Cantidad de coeficientes de cada filtro en la implementación multitasa (entre paréntesis, la del FIR directo equiripple).

f_s	**PB**	**PA**	**PBanda**	**EB**
8 000 Hz	49 (50)	80 (318)	137 (316)	218 (1 108)
16 000 Hz	93 (94)	108 (632)	221 (634)	246 (2 216)
22 000 Hz	129 (128)	132 (870)	287 (872)	270 (3 048)
44 000 Hz	255 (256)	214 (1 744)	523 (1 746)	352 (6 124)
48 000 Hz	279 (278)	230 (1 902)	567 (1 904)	368 (6 680)

Los filtros comparten bloques, así que lo que se guarda en flash no es la suma de la tabla: G_{PA} y G_{EB} son los mismos para las cinco f_s, y H_D es el mismo para el PA, el PBanda y el EB de cada f_s. En total son 2 531 coeficientes distintos, 2 576 con los ceros de relleno, frente a 30 920 del FIR directo. Los valores están en el Anexo **??**. La cantidad de coeficientes tampoco es la cantidad de productos por muestra (tabla 17), porque H_D se reparte entre las M muestras y G se calcula una vez cada M.

La ventaja principal es que el costo del camino angosto ya no depende de f_s: G trabaja siempre a 1 kHz, y lo único que crece con f_s son los filtros de transición ancha (H_D, PB y LPB). Las desventajas son tres. La estructura es más compleja y su retardo es mayor que el del FIR directo equiripple, porque suma el de las dos pasadas por H_D y el de filtros G diseñados con margen. El cambio de tasa agrega componentes espurias, acotadas a $-41{,}8$ dB. Y el sistema ya no es LTI, así que su verificación no se puede limitar a la respuesta en frecuencia.

4 Diseño de la aplicación

4.1 Arquitectura general

La figura 11 muestra las partes de la aplicación. El camino de la señal es el del Laboratorio 1 con un bloque de procesamiento en el medio: el CTIMER0 dispara cada conversión a través del INPUTMUX, el LPADC0 deja el resultado en su FIFO y la rutina de interrupción lo convierte a q15, lo pasa por el filtro activo (o lo copia, en *bypass*) al buffer de salida y desde ahí lo escribe en el DAC0; mientras se ejecuta, mantiene en 1 el pin de medición `P0_26`. Las teclas sólo modifican el estado: SW3 cambia la frecuencia de muestreo (match del CTIMER0) y SW2 el tipo de filtro. Ambas avisan a la rutina del ADC, que es la única que cambia el filtro activo. El lazo principal lee el touchpad, que habilita o deshabilita el filtro, y es el único que escribe el LED: su color indica la frecuencia de muestreo y su parpadeo, el filtro (sección 4.4).

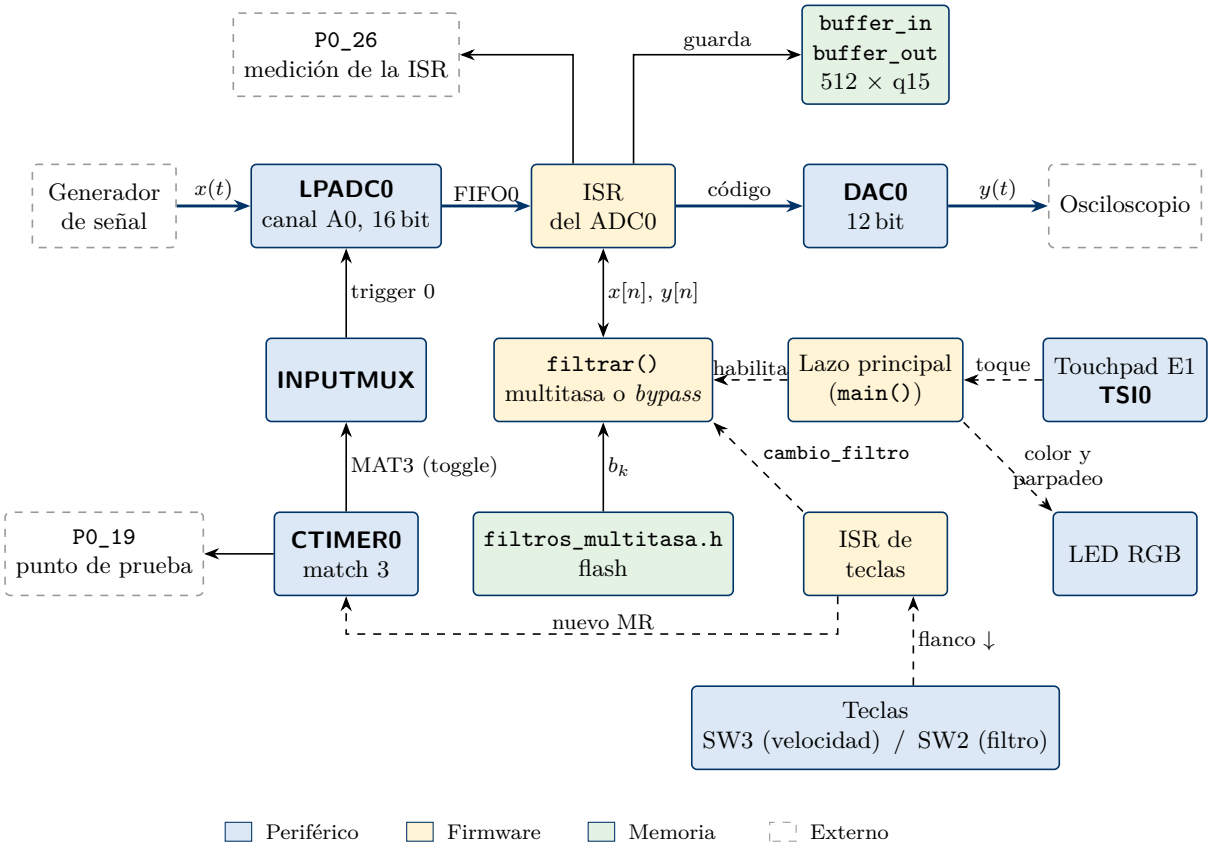


Figura 11: Diagrama de bloques de la aplicación. Trazo continuo azul: señal; negro: datos y disparo; punteado: control.

4.2 Velocidades de muestreo

La generación de las velocidades no cambió respecto del Laboratorio 1: con el timer contando a 150 MHz, reiniciándose en el match y con la salida en toggle,

$$f_s = \frac{f_{\text{timer}}}{2\,(\text{MR} + 1)}. \tag{15}$$

La tabla 14 repite los valores de MR (arreglo `sample_vel`) y el color del LED de cada estado.

Tabla 14: Velocidades de muestreo, valor de match programado y color del LED.

Estado	f_s nominal	MR	f_s real	Error	LED
E0	8 000 Hz	9 374	8 000,00 Hz	0,000 %	Verde
E1	16 000 Hz	4 687	15 998,29 Hz	−0,011 %	Amarillo
E2	22 000 Hz	3 408	22 000,59 Hz	0,003 %	Azul
E3	44 000 Hz	1 704	43 988,27 Hz	−0,027 %	Rojo
E4	48 000 Hz	1 562	47 984,64 Hz	−0,032 %	Blanco

4.3 Máquina de estados

El comportamiento de las teclas se modela con dos variables de estado independientes (figura 12):

- `state_counter`: índice de velocidad (E0 a E4). Avanza en forma circular con SW3. Cada cambio reprograma el match del CTIMER0, actualiza el color del LED y obliga a cambiar los coeficientes del filtro activo por los diseñados para la nueva f_s.
- `filter_counter`: tipo de filtro (F0 a F3). Avanza en forma circular con SW2: PB → PA → PBanda → EB → PB.

Las dos teclas actúan en cualquier estado, y la combinación de ambas variables elige uno de los 20 filtros de la tabla 11. El color del LED indica la velocidad y su frecuencia de parpadeo, el filtro. Al reiniciar la placa el sistema arranca en E0 (8 kS/s), con el PB y con la salida filtrada. Si la salida va filtrada o no lo decide el touchpad, independientemente de estas dos variables (sección 4.4).

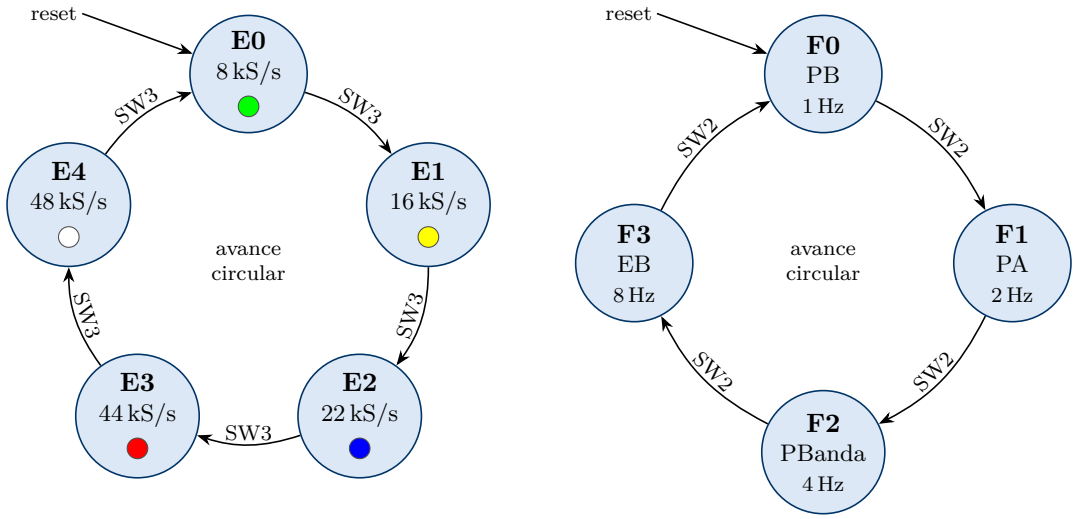


Figura 12: Izquierda: selección circular de la velocidad de muestreo con SW3 y color del LED en cada estado. Derecha: selección circular del filtro con SW2 y frecuencia de parpadeo del LED con la salida filtrada.

4.4 Habilitación del filtro e indicación en el LED

Touchpad. La salida filtrada se habilita y se deshabilita con el touchpad E1 de la placa, un electrodo capacitivo conectado a `P1_3`, que es el canal 3 del TSI0 (*Touch Sensing Input*). El TSI trabaja en modo *self-cap*: carga y descarga la capacidad del electrodo y entrega una cuenta que baja cuando un dedo la aumenta. Cada toque invierte `salida_filtrada`: con 1, la rutina del ADC escribe en el buffer de salida la salida del filtro; con 0 hace *bypass*: copia la muestra del

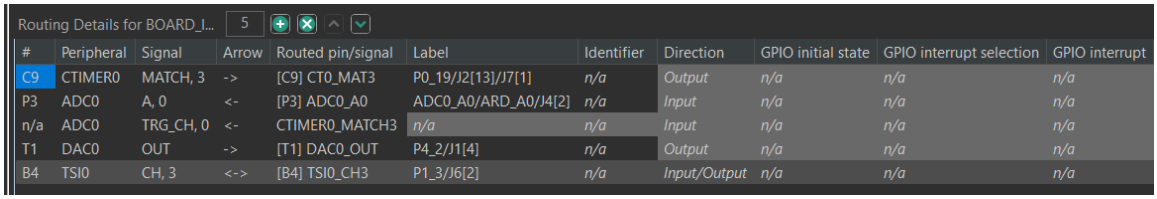
ADC al buffer de salida y no calcula el filtro. En los dos casos el DAC lee del buffer de salida, como pide la consigna.

El touchpad se lee desde el lazo principal, sin interrupciones, para no competir con la rutina del ADC:

- El pin y el reloj se configuran en Config Tools (figura 13): `P1_3` como `TSI0_CH3`, con el buffer de entrada digital deshabilitado porque es una entrada analógica, y el reloj del TSI desde el FRO de 12 MHz (el cristal externo que usa el ejemplo del SDK no está habilitado en este proyecto).
- Al arrancar, `init_touch()` inicializa el TSI en modo self-cap, con disparo por software y sin interrupciones, y mide la *base*: el promedio de 16 barridos sin tocar. Por eso no hay que tocar el touchpad durante el reset.
- Cada 10 ms, `touch_actualizar()` dispara un barrido por software y, en las vueltas siguientes del lazo, consulta si terminó. Nunca espera: si un barrido no termina en 50 ms, el touchpad se desactiva y se avisa por la consola, y el resto del programa sigue funcionando.
- Una lectura cuenta como toque si está más de 20 cuentas por debajo de la base, y como liberación si vuelve a quedar a menos de 10 (histéresis). El cambio se acepta recién con 3 lecturas seguidas (unos 30 ms de antirrebote), y sólo el paso de *sin tocar* a *tocado* invierte la salida: mantener el dedo apoyado no la vuelve a cambiar.
- Al volver a habilitar el filtro se activa `cambio_filtro`, así que la rutina del ADC borra los estados antes de la primera muestra filtrada (sección 4.6.5) y el filtro no arranca con la historia de antes de deshabilitarlo. A la salida se ve el transitorio de $2D$ muestras del PA y del EB.

El umbral de 20 cuentas es el del ejemplo `tsi_v6_self_cap` del SDK. Poniendo `DEPURAR_TOUCH` en 1, el firmware imprime la cuenta cada 200 ms, lo que permite ajustarlo con la placa.

El pin `P1_23`, que se usaba antes para elegir la salida, ya no está asignado en Config Tools, y la instancia GPIO1 de la herramienta Peripherals, que sólo habilitaba su interrupción (`GPIO10_IRQn`), está deshabilitada. El firmware ya no tiene rutina para esa interrupción.


Routing Details for BOARD_I... 5

#	Peripheral	Signal	Arrow	Routed pin/signal	Label	Identifier	Direction	GPIO initial state	GPIO interrupt selection	GPIO interrupt
C9	CTIMER0	MATCH, 3	->	[C9] CT0_MAT3	P0_19/J2[13]/J7[1]	n/a	Output	n/a	n/a	n/a
P3	ADC0	A, 0	<-	[P3] ADC0_A0	ADC0_A0/ARD_A0/J4[2]	n/a	Input	n/a	n/a	n/a
n/a	ADC0	TRG_CH, 0	<-	CTIMER0_MATCH3	n/a	n/a	Input	n/a	n/a	n/a
T1	DAC0	OUT	->	[T1] DAC0_OUT	P4_2/J1[4]	n/a	Output	n/a	n/a	n/a
B4	TSI0	CH, 3	<->	[B4] TSI0_CH3	P1_3/J6[2]	n/a	Input/Output	n/a	n/a	n/a

Figura 13: Config Tools — Pins. `P1_3` asignado a `TSI0_CH3` (touchpad E1), con el buffer de entrada deshabilitado.

LED. El LED RGB conserva el color de cada frecuencia de muestreo (tabla 14), y su parpadeo indica el estado del filtro (tabla 15). La base de tiempo es el SysTick, configurado para interrumpir cada 1 ms con la prioridad mínima. Sólo el lazo principal escribe el LED (`led_actualizar()`), y sólo cuando cambia lo que tiene que mostrar, así que no hay dos rutinas escribiendo los mismos pines.

Tabla 15: Indicación del LED: el color da la f_s y el parpadeo, el filtro.

Estado	LED	Semiperíodo
Salida sin filtrar	Fijo	—
PB	Parpadea a 1 Hz	500 ms
PA	Parpadea a 2 Hz	250 ms
PBanda	Parpadea a 4 Hz	125 ms
EB	Parpadea a 8 Hz	62 ms

La figura 14 resume el lazo principal. Como la rutina del ADC lo interrumpe en cada muestra, el lazo corre en el tiempo que queda libre, que en el peor caso estimado (el PBanda a 48 kS/s) es más de un tercio del período de muestreo; de sobra para tareas que tienen tiempos de decenas de milisegundos.

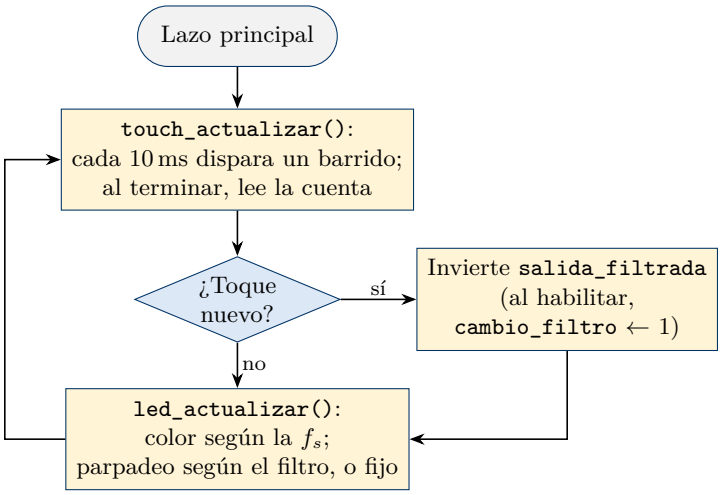


Figura 14: Lazo principal: lectura del touchpad y control del LED.

4.5 Formato q15 y conversiones

Un número q15 es un entero de 16 bits con signo que representa el valor

$$v_{\text{q15}} = \frac{x_{q15}}{2^{15}} \in \left[-1,\ 1 - 2^{-15}\right]. \tag{16}$$

En resolución alta el LPADC entrega un resultado de 16 bit sin signo, $v \in [0,\ 65535]$. Como la entrada es single-ended, v es unipolar; para centrarla en cero basta restar la mitad de escala:

$$x[n] = v - 32768 \in [-32768,\ 32767]. \tag{17}$$

El resultado ocupa exactamente el rango de un `q15_t`: 0 V corresponde a -1, 1,65 V a 0 y 3,3 V a casi $+1$.

A la salida, la muestra filtrada $y[n]$ (en q15) se lleva a los 12 bit del DAC descartando los 4 bits menos significativos y volviendo a sumar la mitad de escala, con saturación:

$$\text{código}_{\text{DAC}} = \text{sat}_{[0,\,4095]}\left(\frac{y[n]}{16} + 2048\right). \tag{18}$$

La saturación es necesaria porque, a diferencia del Laboratorio 1, la salida ya no es una copia de la entrada: un filtro con ganancia mayor a 1 en alguna frecuencia, o el rizado de la banda de paso, puede llevar $y[n]$ fuera del rango.

4.6 Filtrado

4.6.1 Procesamiento muestra a muestra

`arm_fir_q15()` implementa la ecuación (1) sobre un bloque de `blockSize` muestras. Con `BLOCK_SIZE = 1` se llama con una sola muestra, así que la salida se obtiene en la misma interrupción en que llega la entrada y el único retardo agregado es el del propio filtro. La función guarda internamente las últimas N muestras en el buffer de estado, que cumple el papel de línea de retardo. En la implementación multitasa se usa así para el PB y el LPB, una vez por interrupción, y para G, a 1 kHz, una vez cada M interrupciones. El resto del camino angosto (diezmador, interpolador y complemento) está escrito a mano (sección 4.6.3).

Internamente los coeficientes y las muestras son 1.15; cada producto es un 2.30 y se acumula en 64 bits (formato 34.30), de modo que la suma no desborda aunque el filtro tenga miles de coeficientes. Al final el acumulador se trunca a 15 bits fraccionarios y se satura a 1.15. La única pérdida de precisión, además de la cuantización de los coeficientes, ocurre en ese último paso.

4.6.2 Estructura de implementación

`arm_fir_q15()` realiza el cálculo con la estructura directa: una línea de retardo de N muestras y N multiplicaciones por muestra de salida, una por coeficiente. Los cuatro filtros son de fase lineal, así que sus coeficientes son simétricos, $b_k = b_{N-1-k}$. La teoría presenta para ese caso la estructura directa de fase lineal, que suma primero las muestras que comparten coeficiente,

$$y[n] = \sum_{k=0}^{N/2-1} b_k \left(x[n-k] + x[n-N+1+k]\right), \tag{19}$$

y reduce las multiplicaciones a la mitad (N par). `arm_fir_q15()` no aprovecha esa simetría, de modo que con ella se podría bajar la carga de CPU de la sección 4.8. No se implementó porque exige escribir el filtro propio en lugar de usar la biblioteca, y porque la suma previa de dos muestras q15 puede desbordar los 16 bits y requiere trabajar en 32. Las sumas y los accesos a memoria no se reducen, así que el ahorro de ciclos sería menor al de multiplicaciones.

4.6.3 Camino multitasa con el diezmador repartido

El camino angosto de la ecuación (9) está en `complemento_multitasa()` (código 2), que traduce a C el modelo muestra a muestra de `disenar_multitasa.m`.

La forma directa de la ecuación (6) calcula $u[k]$ en el instante de diezmado ($n = kM$), con N_D productos de una sola vez. Como en ese mismo instante también se calcula G, esa muestra concentra casi todo el trabajo: en el PBanda a 48 kS/s serían 573 productos, un 92 % del período de muestreo, cuando el promedio es de 347. En el procesamiento por muestras lo que importa es el pico, porque cada muestra tiene que terminar antes de que llegue la siguiente.

Para evitarlo, el diezmador se calcula *repartido*. Con h_D completado con ceros hasta LM coeficientes ($L = \lceil N_D/M \rceil$), la muestra $x[n]$, de fase $p = n \bmod M$, aparece en la suma de $u[k]$ para todos los k con $0 \le kM - n < LM$. Esos k son exactamente L:

- si $p = 0$: $u[n/M], u[n/M+1], \ldots$, con los coeficientes $h_D[0], h_D[M], \ldots$;
- si $p \neq 0$: $u[\lfloor n/M \rfloor + 1], u[\lfloor n/M \rfloor + 2], \ldots$, con los coeficientes $h_D[M-p], h_D[2M-p], \ldots$

Entonces, en lugar de guardar x y hacer toda la suma al final, cada muestra que llega suma sus L productos a L acumuladores (`acc_hd`, un anillo de L posiciones). Cuando $p = 0$, el acumulador de $u[n/M]$ acaba de recibir su último término: se lee, se pone en cero para reutilizarlo y su valor

pasa por G. El resultado es el mismo $u[k]$ con el mismo retardo D, con $L = 4$ productos por muestra en lugar de 181 cada 48 (a 48 kS/s).

La interpolación de la ecuación (8) se calcula en forma polifásica. De los LM coeficientes de $M\,h_D$, en cada muestra sólo los de la fase p multiplican a valores distintos de cero:

$$v[n] = M \sum_{m=0}^{L-1} h_D[p + mM]\; w\big[\lfloor n/M \rfloor - m\big], \tag{20}$$

así que también cuesta L productos por muestra. Las salidas de G se guardan en `hist_w` (L posiciones, la más nueva primero) y la entrada en `hist_x`, un buffer circular de `LONG_HIST_X` = máx $D + 1$ muestras del que se lee $x[n - D]$.

Los productos se acumulan en 64 bits y se llevan a q15 con un desplazamiento de 15 bits y saturación, como en CMSIS-DSP. La ganancia M del interpolador se aplica sobre el acumulador, antes del desplazamiento: los coeficientes de h_D entran en q15 con margen, mientras que los de $M\,h_D$ llegan a casi 1 y no lo tendrían.

4.6.4 Instancias de los filtros

Los filtros que se calculan con CMSIS-DSP se representan con estructuras `arm_fir_instance_q15` (cantidad de coeficientes, puntero a los coeficientes y puntero al buffer de estado). Al arrancar, `init_filtros()` inicializa `fir_pb[fs]` y `fir_lpb[fs]` a partir de las tablas `pb_tabla`, `pb_taps`, `lpb_tabla` y `lpb_taps`, y `fir_g[0]` y `fir_g[1]` con `G_PA` y `G_EB`. El camino angosto activo se describe con una estructura `camino_angosto_t` (puntero a h_D, M, L, D y la instancia de G), que se completa al cambiar de filtro a partir de las tablas del archivo de cabecera.

Como hay un único filtro activo a la vez, los filtros de cada grupo comparten el buffer de estado: `estado_fs`, dimensionado para el más largo de los PB y LPB (338 coeficientes), y `estado_g` (188). En total el estado ocupa unos 10 KiB de RAM, casi todo en `hist_x` (4 645 muestras, por el EB a 48 kS/s), frente a los 15 210 bytes del buffer de estado de la implementación previa.

4.6.5 Cambio sincronizado de filtro

Las rutinas de las teclas no tocan el filtro directamente: sólo actualizan su variable de estado y ponen en 1 la bandera `cambio_filtro`. La rutina del ADC revisa la bandera antes de procesar cada muestra y, si está activa, llama a `aplicar_seleccion()`, que:

1. fija el tipo de filtro activo, la instancia directa (`fir_pb` o `fir_lpb` de la f_s activa) y los parámetros del camino angosto (h_D, M, L, D y G);
2. borra los buffers de estado y las historias del camino angosto, y pone en cero los contadores de fase, para que el nuevo filtro no arranque con la historia del anterior (que puede haber sido calculada a otra f_s o con otra estructura).

Así el cambio ocurre siempre entre dos muestras y nunca en medio de un cálculo. Como todas las interrupciones tienen la misma prioridad, tampoco puede pasar que una tecla modifique el estado mientras la rutina del ADC lo está leyendo. El borrado del estado produce, después de cada cambio, un transitorio tan largo como la respuesta impulsional del filtro activo ($2D$ muestras en el PA y el EB), durante el cual las líneas de retardo se vuelven a llenar.

4.7 Rutinas de interrupción

La figura 15 muestra la rutina del ADC, que se ejecuta una vez por muestra, y la figura 16 las de las teclas.

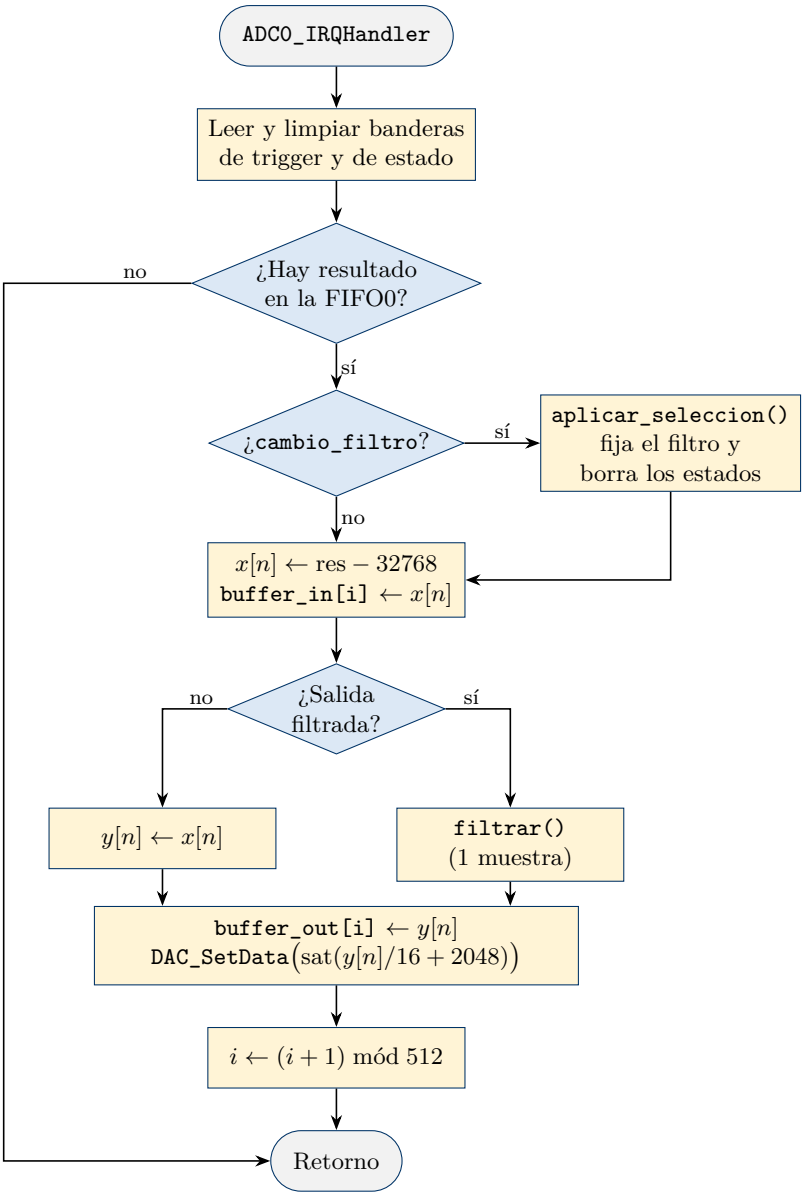


Figura 15: Diagrama de flujo de la rutina de fin de conversión del ADC. El *bypass* también se usa mientras `filtros_listos` es falso (antes de `init_filtros()`).

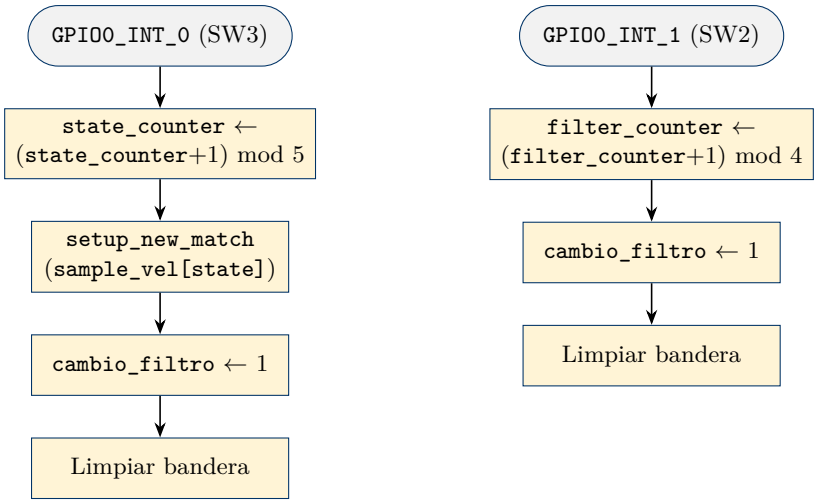


Figura 16: Diagramas de flujo de las rutinas de las teclas. Izquierda: cambio de velocidad (SW3). Derecha: cambio de filtro (SW2). El touchpad y el LED se atienden desde el lazo principal (figura 14).

4.8 Restricción de tiempo real

Todo el procesamiento de una muestra tiene que terminar antes de que llegue la siguiente. Con el núcleo a $f_{\mathrm{CPU}} = 150\,\mathrm{MHz}$, los ciclos disponibles por muestra son f_{CPU}/f_s, y el filtro consume aproximadamente $c\,N$ ciclos, donde c es el costo por coeficiente de `arm_fir_q15()`. La fracción del período de muestreo que ocupa el filtro es entonces

$$U = \frac{c\,N\,f_s}{f_{\mathrm{CPU}}}, \tag{21}$$

y el sistema funciona en tiempo real sólo si $U < 1$ (con algo de margen para el resto de la rutina y las teclas). La tabla 16 aplica la ecuación (21) a los filtros directos de la implementación previa (tabla 6) con $c = 5$ ciclos por coeficiente, la estimación que usa `generar_filtros.m`.

Tabla 16: Implementación previa (FIR directo, ventana de Kaiser): carga estimada de CPU, U [%], con $c = 5$ ciclos por coeficiente. En rojo, las combinaciones que no entran en tiempo real.

f_s	Ciclos/muestra	PB	PA	PBanda	EB
8 000 Hz	18 750	1,7	21,9	21,9	33,9
16 000 Hz	9 375	6,6	87,6	87,5	135,3
22 000 Hz	6 818	12,5	165,4	165,3	255,6
44 000 Hz	3 409	49,9	661,2	661,2	1 022,3
48 000 Hz	3 125	59,2	786,9	786,6	1 216,6

Según esta estimación, a 8 kS/s los cuatro filtros entran con holgura y el PB entra a todas las velocidades, pero el EB supera el período de muestreo a partir de 16 kS/s y el PA y el PBanda a partir de 22 kS/s. La carga crece con f_s^2: al duplicar f_s se duplica la cantidad de coeficientes (sección 3.3.2) y también la cantidad de muestras por segundo. Con el diseño equiripple de orden mínimo (tabla 8) el panorama casi no cambia: entran 12 de las 20 combinaciones en lugar de 10.

Cuando $U > 1$ la rutina del ADC todavía no terminó cuando llega el disparo siguiente. El disparo lo genera el hardware, así que el ADC sigue convirtiendo, pero la interrupción queda pendiente y se atiende recién al salir de la actual: se pierden muestras y la salida se actualiza a una tasa menor que f_s. Además, el lazo principal (touchpad y LED) corre sólo con el tiempo

que deja la rutina del ADC: con U cercano a 1 casi no se ejecuta, el LED deja de parpadear y el cambio de filtro parece no responder. Eso se observó en la placa con la primera versión de la implementación multitasa (sección 4.8.1).

Carga de la implementación multitasa. En la implementación multitasa el costo por muestra ya no es N. En cada muestra de entrada, el diezmador repartido hace L productos y el interpolador otros L, y una de cada M muestras se calcula además G, con N_G productos (contando el cero de relleno). El peor caso por muestra es entonces

$$\mathrm{MAC}_{\mathrm{pico}} = 2L + N_G \quad (+\,N_{\mathrm{LPB}} \text{ en el PBanda}), \tag{22}$$

que no depende de f_s en el PA y el EB (58 y 196 productos), porque G trabaja siempre a 1 kHz y $L = 4$ para las cinco velocidades. La tabla 17 aplica la ecuación (21) con esos productos en lugar de N.

Tabla 17: Implementación multitasa: carga estimada de CPU en la peor muestra, U [%], con $c = 5$ ciclos por producto y el diezmador repartido.

f_s	**Ciclos/muestra**	**PB**	**PA**	**PBanda**	**EB**
8 000 Hz	18 750	1,3	1,5	3,1	5,2
16 000 Hz	9 375	5	3,1	9,2	10,5
22 000 Hz	6 818	9,5	4,3	15,7	14,4
44 000 Hz	3 409	37,5	8,5	54	28,7
48 000 Hz	3 125	44,8	9,3	63,4	31,4

Con $c = 5$ las 20 combinaciones entran en tiempo real y el peor caso es el PBanda a 48 kS/s, con un 63 %, que lo domina el LPB directo (338 coeficientes), no el camino angosto. Sin repartir el diezmador, ese caso llegaría al 92 % en la muestra en que se diezma (sección 4.6.3). Es una estimación: el valor de c real resultó muy distinto, como muestra la sección siguiente.

4.8.1 Carga medida

La rutina del ADC mide su duración con el contador de ciclos del DWT (`MEDIR_CICLOS_ISR` en `source/TP2.c`): guarda el máximo desde el último cambio de filtro o de f_s (`isr_ciclos_max`) y cuenta las veces que duró más que el período de muestreo (`isr_excesos`). Se leen desde el depurador. La tabla 18 resume las medidas en los dos casos con filtro directo a f_s más exigentes, sin señal de entrada (el costo del FIR no depende del dato).

Tabla 18: Ciclos máximos de la rutina del ADC (`-O2`). El presupuesto es f_{CPU}/f_s. La primera versión usa `arm_fir_q15()` con `blockSize = 1`; la segunda, `fir_muestra()`.

Caso	**FIR**	**Máx.**	**Presupuesto**	U **pico**	**Excesos**
PB, 44 kS/s	CMSIS	3 378	3 409	99 %	0
PB, 44 kS/s	propio	1 189	3 409	35 %	0
PBanda, 48 kS/s	CMSIS	5 364	3 125	172 %	85 812
PBanda, 48 kS/s	propio	2 034	3 125	65 %	0

El costo por coeficiente: medición y causa

Con `arm_fir_q15()` el costo medido fue de unos 10 ciclos por coeficiente (3 378/340 en el PB a 44 kS/s), el doble de los $c = 5$ estimados, aun con la biblioteca compilada con `-O2`. La causa está en el código de la biblioteca: con `blockSize = 1` no se usa el lazo SIMD sino el escalar (una carga de 16 bits, un producto y una suma de 64 bits por coeficiente), y al final de cada llamada se copian de a una las $N - 1$ muestras del estado. Con el PB a 44 kS/s la rutina ocupó el 99 % del período; con el PBanda a 48 kS/s excedió el período (hasta el 172 %) y el lazo principal no corrió: el LED se quedaba fijo y el cambio de filtro con el SW2 parecía requerir dos pulsaciones, aunque el cambio se había hecho. Sólo ocurría con el PB y el PBanda, que son los que calculan un FIR largo a f_s.
Se reemplazó esa llamada por `fir_muestra()`, que da el mismo resultado (acumulador de 64 bits, desplazamiento de 15 bits y saturación) con dos cambios: guarda la historia duplicada (`hist[p]` y `hist[p + N]`), de modo que la ventana es contigua y no se copia nada, y usa `__SMLALD`, que hace dos productos por instrucción. Un primer intento con `read_q15x2()` de CMSIS empeoró la rutina al doble (6 771 ciclos en el PB a 44 kS/s): esa función usa `memcpy()`, y como el proyecto compila con `-fno-builtin` cada lectura era una llamada real a la biblioteca. Con una lectura de 32 bits directa el costo bajó a unos 3,5 ciclos por coeficiente, 2,8 veces menos que con CMSIS, y el peor caso quedó en el 65 %.
Esto matiza lo que advierte la teoría sobre usar la biblioteca antes que un lazo propio: vale para tamaños de bloque razonables, pero con una sola muestra por llamada y el SIMD sin usar, un lazo escrito para ese caso es más rápido.

5 Implementación

La lógica propia está en `source/TP2.c` y los coeficientes en `source/filtros_multitasa.h`; el resto del proyecto es código generado por Config Tools y drivers del SDK. El proyecto completo está referenciado en el Anexo A.

5.1 Constantes y variables globales

Los valores de match se mantienen en `sample_vel`, igual que en el Laboratorio 1. `filtros_multitasa.h` define `NUM_FS` (5), `NUM_TIPOS` (4), `F_BAJA_HZ` y las constantes que dimensionan las instancias y los buffers (`MAX_TAPS_FS`, `LONG_HIST_X` y `MAX_HIST_W`). Las variables que comparten las rutinas de interrupción (`state_counter`, `filter_counter`, `salida_filtrada`, `cambio_filtro` y `filtros_listos`) se declaran `volatile`. El estado del camino angosto (`camino`, `hist_x`, `acc_hd`, `hist_w` y los contadores `fase` y `ranura`) sólo lo usa la rutina del ADC, y el del touchpad (`touch_base`, `touch_tocado` y los demás `touch_*`), sólo el lazo principal. `ms_ticks`, que incrementa el SysTick, es la base de tiempo del touchpad y del LED; `semiperiodo_led_ms` guarda el semiperíodo de parpadeo de cada filtro (tabla 15). Los buffers `buffer_in` y `buffer_out` guardan las últimas 512 muestras de entrada y de salida, lo que permite inspeccionarlas con el depurador.

5.2 Programa principal

`main()` inicializa pines y relojes, luego el DAC (`init_dac()`) y el pin de medición (`init_pin_medicion()`) y recién después los periféricos generados por Config Tools, por el motivo explicado en la sección 2.4. A continuación inicializa las instancias de filtro, activa `cambio_filtro` para que la primera muestra aplique la selección inicial, configura el SysTick para interrumpir cada 1 ms, inicializa el touchpad (`init_touch()`, que mide la base) y entra en el lazo principal, que lee el touchpad y actualiza el LED (figura 14).

5.3 Inicialización de los filtros

`init_filtros()` llama a `arm_fir_init_q15()` para las 12 instancias de CMSIS-DSP: el PB y el LPB de cada f_s, con sus tablas y el buffer `estado_fs`, y los dos filtros G, con `estado_g`. Si alguna inicialización falla (por ejemplo, por una cantidad impar de coeficientes), lo informa por la consola de depuración. Al terminar pone `filtros_listos` en 1; hasta ese momento la rutina del ADC no filtra ni aplica cambios de filtro.

5.4 Rutina del ADC

El código 1 muestra el núcleo de la rutina del ADC. Antes de procesar la muestra se aplica, si corresponde, el cambio de filtro pendiente; después se convierte a q15, se filtra (o se copia) y se envía al DAC. Si `MEDIR_TIEMPO_ISR` vale 1, la rutina pone en 1 el pin `P0_26` al entrar y en 0 al salir, para medir su duración (sección 6). `salida_filtrada` la cambia el touchpad desde el lazo principal (sección 4.4).

Código 1: Procesamiento de una muestra en `ADC0_IRQHANDLER()` (`source/TP2.c`).

```
if (LPADC_GetConvResult(ADC0_PERIPHERAL, &result, 0)) {
  if (cambio_filtro) {
    aplicar_seleccion();
  }

  /* ADC de 16 bits: 0..65535 -> Q15 centrado en 0 */
```

```
  buffer_in[index_buffer] = (q15_t)((int32_t)result.convValue - 32768);

  if (salida_filtrada && filtros_listos) {
    buffer_out[index_buffer] = filtrar(buffer_in[index_buffer]);
  } else {
    buffer_out[index_buffer] = buffer_in[index_buffer];      /* bypass */
  }

  /* Q15 -> 12 bits para el DAC */
  int32_t out_dac = ((int32_t)buffer_out[index_buffer] / 16) + ADC_MID;
  if (out_dac < 0)    out_dac = 0;
  if (out_dac > 4095) out_dac = 4095;
  DAC_SetData(DAC0, (uint16_t)out_dac);

  index_buffer = (index_buffer + 1) % BUFFER_SIZE;
}
```

`filtrar()` elige según el tipo activo: el PB es una llamada a `fir_muestra()` (FIR de una muestra con los coeficientes de la instancia de CMSIS, sección 4.8.1); el PA y el EB, una llamada a `complemento_multitasa()`; y el PBanda, las dos en cascada (con el LPB). El código 2 es el camino angosto con el diezmador repartido de la sección 4.6.3.

Código 2: Camino angosto: `complemento_multitasa()` (`source/TP2.c`).

```
static q15_t complemento_multitasa(q15_t x) {
  const camino_angosto_t *c = &camino;
  const uint16_t M = c->m;
  const uint16_t L = c->l;

  pos_x = (pos_x + 1 == LONG_HIST_X) ? 0 : pos_x + 1;
  hist_x[pos_x] = x;

  /* x[n] aporta a los L diezmados pendientes (HD repartido) */
  uint16_t j = (fase == 0) ? 0 : M - fase;
  uint16_t r = (fase == 0) ? ranura : ((ranura + 1 == L) ? 0 : ranura + 1);
  for (uint16_t i = 0; i < L; i++) {
    acc_hd[r] += (int32_t)c->hd[j] * x;
    j += M;
    r = (r + 1 == L) ? 0 : r + 1;
  }

  /* Instante de diezmado: u[n/M] esta completo y pasa por G */
  if (fase == 0) {
    q15_t u = (q15_t)__SSAT((int32_t)(acc_hd[ranura] >> 15), 16);
    q15_t w;
    acc_hd[ranura] = 0;
    w = fir_muestra(c->g, hist_g, &pos_g, u);
    for (uint16_t m = L - 1; m > 0; m--) {
      hist_w[m] = hist_w[m - 1];
    }
    hist_w[0] = w;
  }

  /* Interpolador polifasico: v[n] = M * sum_m HD[p + mM] w[n/M - m] */
  int64_t acc = 0;
  const q15_t *h = &c->hd[fase];
  for (uint16_t m = 0; m < L; m++) {
    acc += (int32_t)h[m * M] * hist_w[m];
  }
  int32_t v = __SSAT((int32_t)((acc * M) >> 15), 16);

  if (++fase == M) {
    fase = 0;
```

```
    ranura = (ranura + 1 == L) ? 0 : ranura + 1;
  }

  int32_t k = (int32_t)pos_x - c->d;
  if (k < 0) {
    k += LONG_HIST_X;
  }
  return (q15_t)__SSAT((int32_t)hist_x[k] - v, 16);
}
```

5.5 Rutinas de las teclas y cambio de velocidad

La rutina de SW3 avanza `state_counter` en forma circular y reprograma el CTIMER0 con `setup_new_match()`, igual que en el Laboratorio 1, y además activa `cambio_filtro` para que se carguen los coeficientes de la nueva f_s. A diferencia del Laboratorio 1, no toca el LED: el lazo principal ve el nuevo `state_counter` y cambia el color con `set_state()`. La rutina de SW2 avanza `filter_counter` módulo 4 y activa la misma bandera; el lazo principal cambia también la frecuencia de parpadeo. El touchpad no tiene rutina de interrupción: se lee desde el lazo principal (sección 4.4).

La macro `FILA_FILTRO()` permite, poniendo `SOLO_8K` en 1, usar siempre los coeficientes diseñados para 8 kS/s sin importar la velocidad activa. Sirve para observar cómo se desplazan en frecuencia las bandas de un filtro digital cuando se lo usa a una f_s distinta de la de diseño: todas las frecuencias características escalan en proporción a f_s. En la implementación multitasa la fila de 8 kS/s también fija $M = 8$, así que la tasa baja pasa a ser $f_s/8$ y el camino angosto escala igual que el resto del filtro.

6 Verificación

6.1 Procedimiento

1. Conectar el generador a `J4[2]` (ADC0_A0) con una senoidal de 1 V y un *offset* que la mantenga entre 0 y 3,3 V.
2. Canal 1 del osciloscopio en la entrada, canal 2 en `J1[4]` (DAC0_OUT).
3. Con $f_s = 8\,\text{kS/s}$, observar la señal sin filtrar (tocando el touchpad E1, con el LED fijo) y recorrer los cuatro filtros con SW2, verificando la frecuencia de parpadeo del LED (tabla 15); para cada filtro, barrer la frecuencia del generador midiendo la amplitud de la salida. Calcular la ganancia $|H| = 20\log_{10}(V_{\text{out}}/V_{\text{in}})$ y compararla con la respuesta diseñada.
4. Repetir para otras f_s y verificar que la respuesta en hertz se mantiene (o, si no entra en tiempo real, documentar qué ocurre).
5. Medir el tiempo de ejecución de la rutina del ADC con el pin de medición `P0_26` (`J2[10]`): con `MEDIR_TIEMPO_ISR` en 1, la rutina lo pone en 1 al entrar y en 0 al salir, así que el ancho del pulso es su duración. Conectar el canal 2 del osciloscopio a ese pin y medir el ancho en la peor muestra, que en los filtros multitasa es la de diezmado (el pulso más ancho de cada grupo de M). La cantidad de ciclos es el ancho por $f_{\text{CPU}} = 150\,\text{MHz}$, y el costo por producto, la diferencia con la rutina sin filtrar dividida por la cantidad de productos de la ecuación (22).

6.2 Verificación por simulación

Antes de las mediciones en la placa se verificó el código de filtrado en la PC. Las funciones de filtrado de `source/TP2.c`, sin modificar, se compilaron con `gcc` junto con `filtros_multitasa.h` y una versión mínima de `arm_fir_q15()` con la misma aritmética que la de CMSIS-DSP (acumulador de 64 bits, desplazamiento de 15 bits y saturación). Para los 20 filtros se procesaron 30 000 muestras de ruido, más del doble de la respuesta impulsional más larga, y la salida se comparó con el modelo de referencia en punto flotante con los mismos coeficientes q15. La diferencia máxima fue de 2,6 LSB, que es lo que aportan los redondeos a q15 del diezmador, de G y del interpolador. Un error de índice en el polifásico o en D habría producido diferencias del orden de la señal.

6.3 Respuesta en frecuencia medida

Las frecuencias de prueba se eligen alrededor de los bordes de la consigna: 50 Hz (centro del EB y comienzo de la banda de paso del PA), unos 20 Hz (rechazo del PA y del PBanda), 35 Hz (corte del PA y del PBanda, cerca de $-6\,\text{dB}$), 1 kHz (banda de paso de los cuatro filtros), y 3,5 kHz y 3,9 kHz (corte del PBanda y rechazo del PB). Para las frecuencias de pocas decenas de hertz hay que usar acoplamiento DC en el osciloscopio: el acoplamiento AC tiene un corte de algunos hertz que falsearía la medición.

Tabla 19: Ganancia medida a $f_s = 8\,\text{kS/s}$.

f [Hz]	PB [dB]	PA [dB]	PBanda [dB]	EB [dB]
20	-0.42	-28.66	-31.42	0
35	-0.42	-5.84	-5.66	-7.21
50	-0.23	-4.82	-8.64	-31.48
1 000	-0.54	-0.27	-0.30	0.06
3 500	-0.21	0.12	-5.39	0.12
3 900	-28.43	0.12	-29	0.15

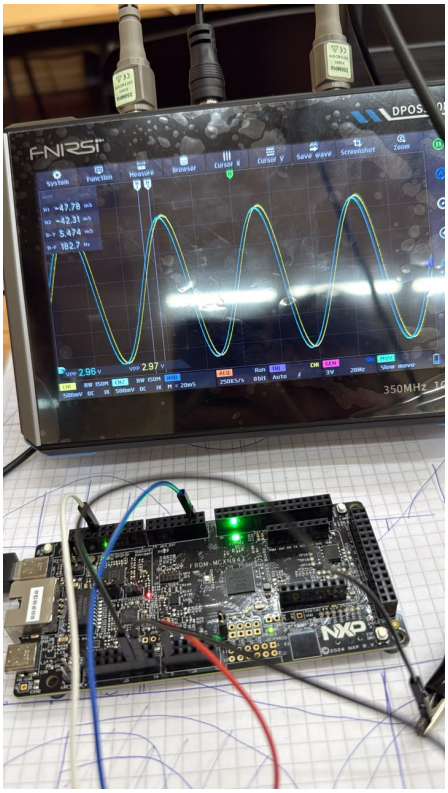


Figura 17: Osciloscopio. Entrada (canal 1) y salida del DAC (canal 2) en *bypass*.

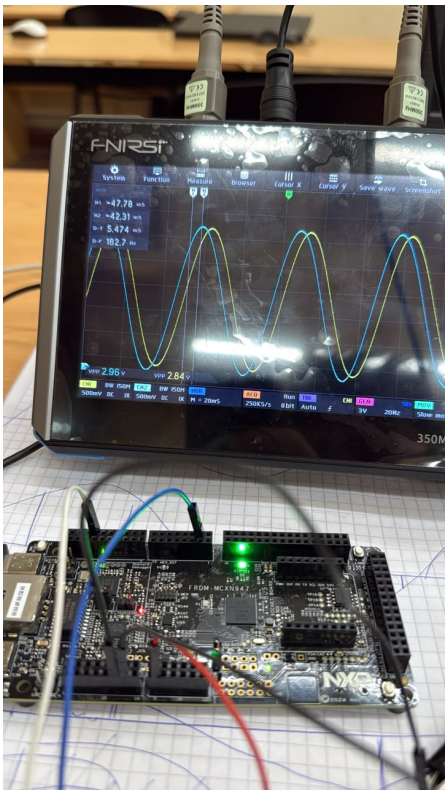


Figura 18: Osciloscopio. Entrada y salida con el filtro pasa bajos y señal de entrada de 20 Hz.

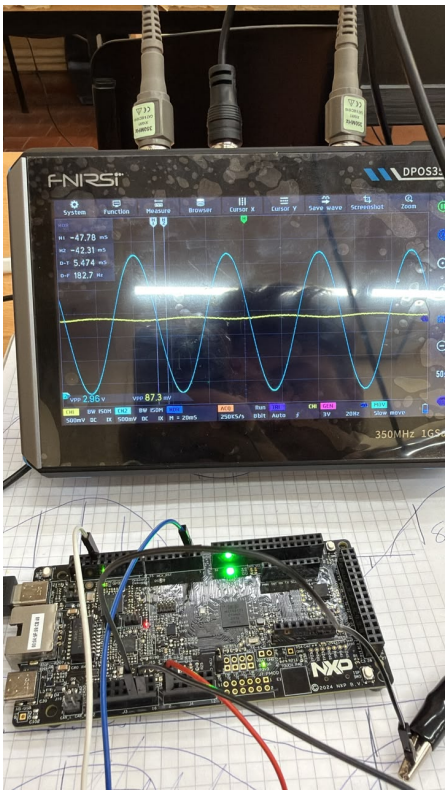


Figura 19: Osciloscopio. Entrada y salida con el filtro pasa altos y señal de entrada de 20 Hz.

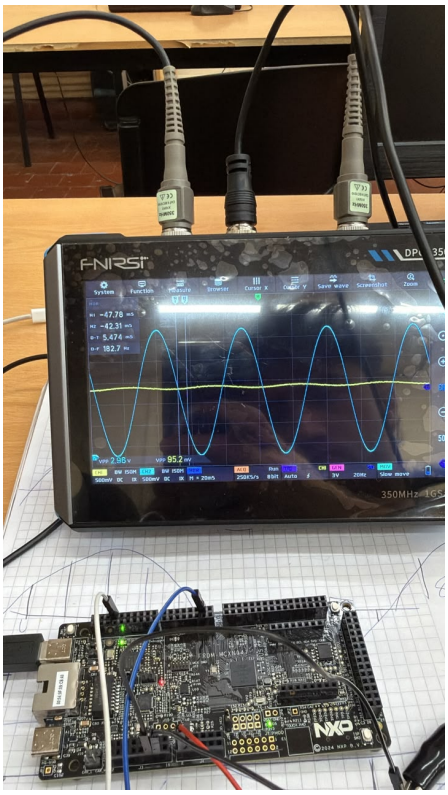


Figura 20: Osciloscopio. Entrada y salida con el filtro pasa banda y señal de entrada de 20 Hz.

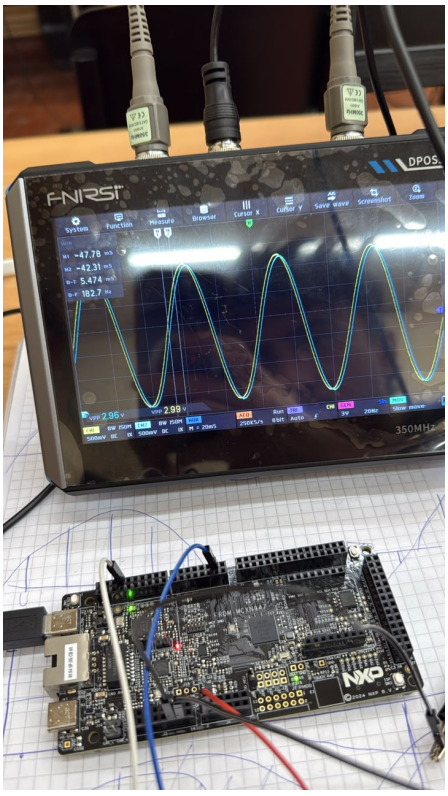


Figura 21: Osciloscopio. Entrada y salida con el filtro elimina banda y señal de entrada de 20 Hz.

6.4 Cumplimiento de los requerimientos

Tabla 20: Verificación de los requerimientos de la consigna.

Req.	Cómo se implementa	Evidencia	Resultado
R1	Cadena CTIMER0 + INPUTMUX + LPADC0 del Laboratorio 1; SW3 elige la f_s; `buffer_in[512]`	Tabla 14	✓
R2	20 filtros (4 tipos × 5 f_s): PB directo y PA, PBanda y EB multitasa, diseñados y verificados en q15 con `disenar_multitasa.m`	Tablas 4 y 11	✓
R3	Muestra a muestra, de `buffer_in` a `buffer_out`: `arm_fir_q15()` con `BLOCK_SIZE = 1` y el camino multitasa	Tabla 19	✓
R4	El touchpad E1 (TSI0) alterna entre filtrar y hacer *bypass* (copia de `buffer_in` a `buffer_out`); SW2 elige el filtro (`filter_counter` módulo 4) y el LED lo indica con su parpadeo	Demostración	✓
R5	`DAC_SetData()` con la muestra de `buffer_out` en cada interrupción del ADC	Figura 17	✓
R6	Entrada y salida en los canales 1 y 2 del osciloscopio	Figuras 18 a 21	✓

7 Conclusiones

Se implementó sobre la FRDM-MCXN947 una aplicación que filtra en tiempo real la señal adquirida por el LPADC0 con filtros FIR pasa bajos, pasa altos, pasa banda y elimina banda, diseñados en MATLAB y ejecutados muestra a muestra en aritmética q15, con `arm_fir_q15()` de CMSIS-DSP y una estructura multitasa para las bandas angostas. Con SW3 se recorren las cinco frecuencias de muestreo del Laboratorio 1, con SW2 se elige uno de los cuatro filtros y con el touchpad E1 se pasa de la señal filtrada a la señal sin filtrar, mientras el LED indica la frecuencia de muestreo con su color y el filtro con su parpadeo; para cada combinación se usan coeficientes diseñados para la f_s activa.

Las decisiones principales del diseño fueron:

- **Procesamiento multitasa para las bandas angostas.** El PA, el PBanda y el EB resuelven sus transiciones de pocos hertz con filtros a 1 kHz, después de diezmar, y vuelven a f_s por interpolación; el PA y el EB se obtienen por complemento. Con eso las 20 combinaciones entran en tiempo real (peor caso estimado: 63 %), con 2 576 coeficientes en lugar de 51 220. El precio es una estructura más compleja, un retardo mayor que el del FIR directo equiripple y componentes espurias por el cambio de tasa, acotadas a −41,8 dB. La estructura se apoya en la misma descomposición polifásica que la FFT de diezmado en el tiempo; la diferencia es que el diezmado descarta fases y por eso necesita el filtro antialias (sección 3.4.4).
- **Respuesta en hertz independiente de f_s**: el diezmador, el PB y el LPB se diseñan para cada velocidad; los filtros angostos son los mismos para las cinco.
- **Diseño equiripple de orden mínimo verificado en q15**, por script, con el reparto del error entre los bloques explícito y la verificación del filtro compuesto, incluido el aliasing.
- **Diezmador repartido** entre las M muestras, para que la muestra de diezmado no concentre el cálculo (en el peor caso, del 92 % al 63 % del período).
- **ADC en** 16 bit, cuyo resultado, centrado, es directamente un `q15_t`.
- **Buffers de estado compartidos** por grupo de filtros, en lugar de uno por filtro.
- **Cambio de filtro diferido a la rutina del ADC**, con borrado del estado, para que nunca se cambie de filtro en medio de un cálculo.
- **Touchpad y LED en el lazo principal**, sin interrupciones propias: el touchpad se lee por *polling* sin esperas, con antirrebote e histéresis, y el LED indica la f_s con su color y el filtro con su parpadeo. Así la rutina del ADC no comparte su prioridad con tareas que no tienen requisitos de tiempo real.

La implementación previa mostró que el método de diseño no es la causa de fondo de la carga de CPU. Según las ecuaciones (4) y (5), con cualquier método el orden de un FIR es proporcional a $f_s/\Delta f$. Las especificaciones del PA, del PBanda y del EB tienen transiciones de pocos hertz, y con f_s de decenas de kilohertz eso exige miles de coeficientes por muestra. El diseño equiripple de orden mínimo redujo los coeficientes entre un 13 % y un 23 % respecto de Kaiser, pero hacía falta un factor de 10 o más. Ese factor lo dio el cambio de estructura: procesar las bandas angostas a 1 kHz. Quedan como mejoras posibles:

- **Aprovechar la simetría de los coeficientes** en el PB y el LPB, que son los que dominan la carga a f_s altas: sumar primero las muestras que comparten coeficiente, $x[n-k]+x[n-N+1+k]$, reduce las multiplicaciones a la mitad. CMSIS-DSP no ofrece esta variante en q15, así que habría que implementarla.
- **Procesar por bloques**, si la consigna lo permitiera: con `blockSize` mayor que 1, `arm_fir_q15()` usa el lazo SIMD de cuatro salidas y copia el estado una vez por bloque, y la carga pasa a ser la promedio en lugar de la de la peor muestra.
- **Compilar con optimización** (`-O2` o `-O3`) para bajar los ciclos por producto.

A Código fuente

El proyecto completo de MCUXpresso (incluidos los archivos generados por Config Tools, los drivers del SDK, CMSIS-DSP y `source/filtros_multitasa.h`) está en la carpeta `TP2/` del repositorio, y los scripts de MATLAB (`disenar_multitasa.m` y `generar_filtros.m`, de la implementación previa) en `documentacion/TP2/src/`. El repositorio es `https://github.com/Mati-Costamagna/procesamiento-digital-de-senales`.